



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Clasificación de TDAH a
partir de señales EEG
mediante técnicas de
Machine Learning y Deep
Learning
Documentación Técnica**



Presentado por Adrián Jiménez García
en Universidad de Burgos — 16 de septiembre
de 2026

Tutor: Bruno Baruque Zanon
Cotutor: Jesús Enrique Sierra García

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	9
Apéndice B Especificación de Requisitos	17
B.1. Introducción	17
B.2. Objetivos generales	18
B.3. Catálogo de requisitos	18
B.4. Casos de uso principales	21
B.5. Requisitos no funcionales	32
Apéndice C Especificación de diseño	35
C.1. Introducción	35
C.2. Diseño de datos	35
C.3. Diseño arquitectónico	43
C.4. Diseño procedimental	49
Apéndice D Documentación técnica de programación	55
D.1. Introducción	55
D.2. Estructura de directorios	55

D.3. Manual del programador	56
D.4. Compilación, instalación y ejecución del proyecto	63
D.5. Pruebas del sistema	68
Apéndice E Documentación de usuario	73
E.1. Introducción	73
E.2. Requisitos de usuarios	73
E.3. Instalación	74
E.4. Manual del usuario	75
Apéndice F Anexo de sostenibilización curricular	85
F.1. Introducción	85
F.2. Sostenibilidad social y ética	85
F.3. Sostenibilidad tecnológica	86
F.4. Sostenibilidad económica y reutilización	86
F.5. Sostenibilidad ambiental	87
F.6. Reflexión final	87
Bibliografía	89

Índice de figuras

A.1. Vista del tablero de ZenHub con tareas en curso y cerradas, y sus pull requests asociados.	3
B.1. Casos de uso del visitante y del usuario autenticado.	22
C.1. Esquema de usuarios, acceso a datasets, experimentos, modelos entrenados y trabajos en segundo plano.	37
C.2. Componentes de la aplicación: API, frontend, worker, Redis, PostgreSQL y ficheros utilizados.	44
C.3. Organización del backend por dominios funcionales y módulos compartidos.	45
C.4. Rutas, vistas, hooks y comunicación con la API en el frontend TypeScript.	47
C.5. Configuración de despliegue con Docker Compose, Cloudflare Tunnel y landing independiente en Cloudflare Pages.	49
C.6. Entrenamiento asíncrono con Celery y consulta periódica de su estado desde el frontend.	51
D.1. Documentación interactiva de la API, incluidos los endpoints de autenticación, trabajos asíncronos y datasets.	59
D.2. Resultado de las pruebas en CI: 144 pruebas superadas e informe de cobertura de backend y scripts.	69
D.3. Ejecución correcta de los trabajos frontend y test en GitHub Actions para el commit b118ff0.	70
D.4. Quality Gate superado y métricas de código nuevo en SonarQube Cloud.	71
E.1. Pantalla de inicio de sesión.	75
E.2. Perfil de la cuenta utilizada en la demostración.	76

E.3. Dataset analizado: 121 sujetos, 19 canales EEG y distribución de clases calculada por filas del CSV.	77
E.4. Configuración de un entrenamiento con XGBoost y características temporales y espectrales.	78
E.5. Estado de ejecución del entrenamiento en segundo plano.	79
E.6. Resultados de XGBoost: métricas, matriz de confusión y confirmación del guardado del modelo.	79
E.7. Importancia por permutación de características y canales sobre 80 ventanas del conjunto de prueba del primer pliegue.	80
E.8. Historial de experimentos y selección del mejor modelo propio disponible.	81
E.9. Información y métricas del modelo de referencia de Machine Learning, Random Forest.	82
E.10. Validación del CSV de un sujeto con el XGBoost del experimento 5 seleccionado.	83
E.11. Resultado de inferencia con XGBoost: clase asignada, confianza media y votos de las 13 ventanas analizadas.	84

Índice de tablas

A.1. Sprints retrospectivos del proyecto.	4
A.2. Ampliaciones posteriores a la primera versión.	5
A.3. Evolución del repositorio por meses.	5
A.4. Ramas principales utilizadas durante el desarrollo.	6
A.5. Planificación temporal del proyecto.	7
A.6. Riesgos principales del proyecto y medidas adoptadas.	9
A.7. Coste mensual estimado del perfil de desarrollo.	10
A.8. Coste de personal imputado al proyecto.	10
A.9. Coste de hardware imputado con amortización lineal.	11
A.10. Coste de software del proyecto.	11
A.11. Coste de infraestructura del proyecto.	12
A.12. Resumen de costes del proyecto.	12
A.13. Licencias de los principales componentes utilizados.	13
B.1. Catálogo de requisitos funcionales de los módulos de modelo, dataset y entrenamiento.	19
B.2. Catálogo de requisitos funcionales de los módulos de experimen- tos, predicción e interfaz.	20
B.3. Requisitos incorporados en la ampliación de la aplicación.	21
B.4. CU-01 Consultar información del modelo de inferencia.	23
B.5. CU-02 Cargar un dataset de entrenamiento.	24
B.6. CU-03 Analizar el dataset y filtrar pacientes.	25
B.7. CU-04 Configurar y ejecutar un entrenamiento.	26
B.8. CU-05 Consultar el histórico de experimentos.	27
B.9. CU-06 Validar el CSV de un paciente.	28
B.10. CU-07 Ejecutar predicción y revisar resultado.	29
B.11. CU-08 Cambiar el modelo activo para inferencia.	30
B.12. CU-09 Cambiar el idioma de la interfaz.	31

B.13.CU-10 Registrar una cuenta e iniciar sesión.	32
C.1. Diccionario de datos de la tabla <code>datasets</code>	38
C.2. Diccionario de datos de la tabla <code>experiments</code>	39
C.3. Diccionario de datos de la tabla <code>experiment_folds</code>	40
C.4. Diccionario de datos de la tabla <code>users</code>	40
C.5. Diccionario de datos de la tabla <code>dataset_accesses</code>	41
C.6. Diccionario de datos de la tabla <code>trained_models</code>	42
C.7. Diccionario de datos de la tabla <code>background_jobs</code>	43

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este apéndice se documenta la planificación temporal del proyecto, la forma de trabajo seguida durante el desarrollo y el estudio de viabilidad económica y legal.

El proyecto consiste en el desarrollo de un clasificador de señales EEG para el apoyo al análisis de TDAH mediante modelos de Machine Learning y Deep Learning. El resultado final es una aplicación web que permite cargar datasets EEG, entrenar modelos, consultar experimentos guardados y realizar inferencia sobre CSV de pacientes individuales. Además, se ha desarrollado un pipeline de investigación para entrenar, evaluar y exportar los modelos utilizados por la aplicación.

El producto entregado incluye:

- Una API REST desarrollada con FastAPI [6].
- Una SPA desarrollada con React y Vite [10, 20].
- Una base de datos PostgreSQL para almacenar datasets y experimentos [12].
- Un pipeline de investigación reproducible en `scripts/`.
- Modelos finales exportados para inferencia.
- Análisis de datasets y entrenamientos en segundo plano mediante Celery y Redis.

- Registro de usuarios, inicio de sesión y acceso separado a sus recursos.
- Configuración de publicación mediante Cloudflare Tunnel y una landing independiente.
- Pruebas automatizadas, integración continua con GitHub Actions y análisis de calidad con SonarCloud [7, 17].

A.2. Planificación temporal

Metodología de trabajo

El desarrollo se ha llevado a cabo de forma incremental. Al tratarse de un TFG con una parte de investigación y otra de desarrollo software, no se ha seguido un modelo en cascada estricto. Primero fue necesario validar que el enfoque metodológico era viable con los datos EEG disponibles y, a partir de ahí, se fue construyendo la aplicación web alrededor del pipeline.

La planificación se organizó en fases cortas, revisando periódicamente los resultados obtenidos. Esta forma de trabajo permitió ajustar el alcance cuando se detectaron necesidades nuevas, como la persistencia de experimentos, la separación entre modelos de Machine Learning y Deep Learning o la inclusión de validación *cross-subject* para evitar fuga de información entre pacientes.

Herramientas de gestión y control de versiones

Para organizar el trabajo se utilizó GitHub como repositorio principal y ZenHub como herramienta de apoyo para la planificación de tareas [8, 21]. La gestión se planteó como un tablero Kanban, más adecuado para un proyecto individual que una metodología Scrum completa. Aun así, el avance se agrupó en bloques temporales similares a sprints, cada uno centrado en un objetivo verificable.

El tablero se organizó mediante columnas sencillas:

- **New Issues e Icebox:** tareas nuevas e ideas sin prioridad inmediata.
- **Product Backlog:** tareas pendientes de planificar.
- **Sprint Backlog:** tareas seleccionadas para el bloque de trabajo.
- **In Progress:** tareas en desarrollo.

- **Review/QA:** cambios implementados pendientes de prueba o revisión.
- **Done:** tareas terminadas e integradas en el repositorio.

Aunque el proyecto fue desarrollado por una sola persona, el tablero permitió mantener una visión ordenada del estado del trabajo y separar tareas de investigación, backend, frontend, pruebas, documentación y ajustes finales.

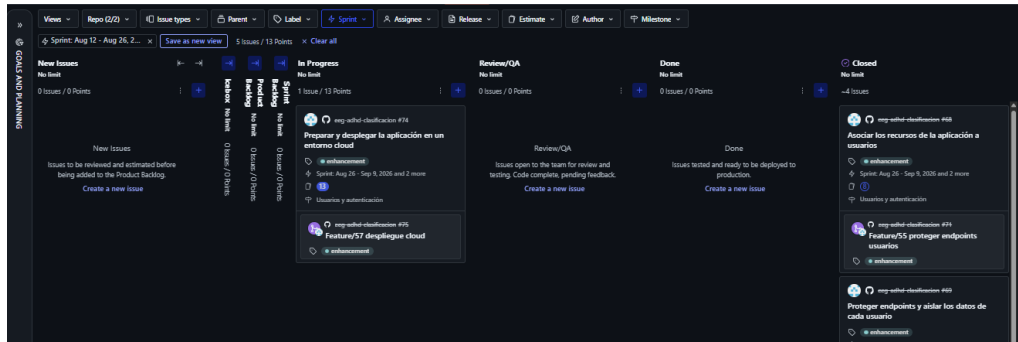


Figura A.1: Vista del tablero de ZenHub con tareas en curso y cerradas, y sus pull requests asociados.

El control de versiones se realizó con Git [16]. En el historial local revisado hasta el commit `b118ff0` se cuentan 171 commits sin incluir merges, entre febrero y agosto de 2026. Durante el desarrollo se utilizaron ramas de funcionalidad para separar prototipos y cambios relevantes, destacando `feature/streamlit-app` y `feature/react-fastapi-app`, antes de consolidar el trabajo final en `main`.

Sprint	Periodo	Objetivo principal	Commits
Sprint 0	26 feb.–9 mar.	Inicio del repositorio, recopilación de artículos y preparación inicial del dataset.	6
Sprint 1	18 mar.–27 mar.	Carga del dataset, preprocesado, división por paciente, generación de epochs y primeras características.	5
Sprint 2	6 abr.–21 abr.	Features espectrales, validación cruzada, modelos de Machine Learning y primeros modelos de Deep Learning.	6
Sprint 3	30 abr.–10 may.	Exportación del mejor modelo ML y primer prototipo de aplicación con inferencia.	4
Sprint 4	14 may.–19 may.	Migración a React + FastAPI, Docker, reorganización de interfaz, CI inicial y pruebas.	14
Sprint 5	24 may.–1 jun.	Robustez de la API, SonarCloud, base de datos de experimentos e histórico.	12
Sprint 6	5 jun.–13 jun.	Internacionalización, mejoras de interfaz, documentación interna y ajustes de inferencia.	8
Sprint 7	18 jun.–21 jun.	Persistencia de datasets, modelos finales, ampliación de tests y documentación.	4

Tabla A.1: Sprints retrospectivos del proyecto.

La división anterior es retrospectiva y recoge la primera etapa del proyecto. Los recuentos se han comprobado sobre el historial local, sin incluir merges. Estos bloques no sustituyen las fechas de los sprints del tablero ni equivalen a horas trabajadas. En mayo se concentró la migración a la aplicación web, Docker, la base de datos y las primeras pruebas automatizadas.

Después se amplió la aplicación para cerrar el flujo entre entrenamiento e inferencia, ejecutar los trabajos en segundo plano y permitir el acceso con usuarios. La siguiente tabla resume esa ampliación a partir de los commits.

Periodo	Trabajo registrado
4–14 de julio	Registro de modelos entrenados, selección para inferencia, refactor del backend y mejora del detalle de experimentos.
15–20 de julio	Celery y Redis, análisis y entrenamiento en segundo plano, migración a uv y Python 3.13.
22–27 de julio	Migración a TypeScript, organización del frontend por funcionalidades y revisión de la interfaz.
29 de julio–3 de agosto	Usuarios, JWT, aislamiento de recursos, registro, login y perfil en la interfaz.
5–9 de agosto	Configuración de Caddy y Cloudflare Tunnel, favicon y landing.

Tabla A.2: Ampliaciones posteriores a la primera versión.

Mes	Commits	Actividad principal
Febrero 2026	1	Inicio del repositorio.
Marzo 2026	10	Dataset, revisión inicial, carga de datos, epochs y primeras features.
Abril 2026	7	Validación cruzada, features espectrales, modelos ML y primeros modelos DL.
Mayo 2026	27	Aplicación web, Docker, backend, frontend, experimentos, CI y pruebas.
Junio 2026	15	Internacionalización, persistencia de datasets, modelos finales, tests y documentación.
Julio 2026	94	Modelos propios, refactors, tareas en segundo plano y autenticación.
Agosto 2026	17	Aislamiento de recursos, acceso desde la interfaz y despliegue.
Total	171	Commits sin merges del historial revisado.

Tabla A.3: Evolución del repositorio por meses.

Rama	Uso principal
main	Rama principal con la versión integrada del proyecto.
feature/streamlit-app	Prototipo inicial de inferencia e interfaz sencilla.
feature/react-fastapi-app	Migración hacia la arquitectura final con React y FastAPI.

Tabla A.4: Ramas principales utilizadas durante el desarrollo.

Fases del proyecto

La Tabla A.5 resume las fases principales del desarrollo. La estimación inicial era de 340 horas. Tras la ampliación de la aplicación, se añaden 50 horas estimadas, hasta un total de 390 horas. Se trata de una estimación del esfuerzo, no de un registro exacto de horas trabajadas.

Fase	Trabajo realizado	Horas
Estudio inicial	Revisión del problema, análisis del dataset EEG, definición del alcance y revisión de trabajos relacionados.	40 h
Pipeline de investigación	Carga de datos, validación del CSV, preprocesamiento, segmentación en epochs y extracción de características.	60 h
Entrenamiento y evaluación	Entrenamiento de modelos ML y DL, validación <i>cross-subject</i> , comparación de métricas y selección de modelos finales.	70 h
Backend y base de datos	Desarrollo de la API con FastAPI, modelos de datos, migraciones, endpoints de inferencia, entrenamiento y experimentos.	55 h
Frontend	Desarrollo de la interfaz en React: vistas de dataset, entrenamiento, experimentos, modelo e inferencia.	55 h
Calidad y despliegue local	Pruebas automatizadas, cobertura, GitHub Actions, SonarCloud, Docker Compose y ajustes de reproducibilidad [13, 14, 5].	30 h
Documentación y memoria	Redacción de la memoria, anexos técnicos, diagramas, capturas y preparación de la defensa.	30 h
Ampliación posterior	Trabajo adicional de ampliación y revisión del proyecto.	50 h
Total estimado	Estimación inicial más la ampliación.	390 h

Tabla A.5: Planificación temporal del proyecto.

Planificación por entregables

El proyecto se puede dividir también según los entregables obtenidos al final de cada etapa:

- **Entregable 1: análisis del problema y dataset.** Se definió el objetivo de clasificación ADHD/Control y se comprobó la estructura de los datos EEG.

- **Entregable 2: pipeline experimental.** Se implementó el procesamiento de la señal, la segmentación en ventanas y la evaluación de modelos.
- **Entregable 3: modelos finales.** Se seleccionaron y exportaron los mejores modelos de Machine Learning y Deep Learning.
- **Entregable 4: aplicación web.** Se desarrollaron backend, frontend y persistencia de experimentos.
- **Entregable 5: ampliación de la aplicación.** Se incorporaron modelos propios para inferencia, tareas en segundo plano, usuarios y aislamiento de recursos.
- **Entregable 6: verificación, despliegue y documentación.** Se añadieron pruebas, integración continua, análisis de calidad, configuración de publicación, diagramas y anexos.

Riesgos y medidas adoptadas

Durante el desarrollo se identificaron varios riesgos técnicos y metodológicos. La Tabla [A.6](#) resume los más relevantes.

Riesgo	Impacto	Medida adoptada
Fuga de información entre pacientes	Métricas demasiado optimistas y poco fiables.	Uso de validación <i>cross-subject</i> con grupos por paciente.
Coste computacional de los modelos DL	Entrenamientos lentos o inestables.	Modelos ligeros, early stopping y parámetros ajustables desde la interfaz.
CSV con formato incorrecto	Fallos durante entrenamiento o inferencia.	Validadores de columnas, canales EEG y etiquetas esperadas.
Dependencia de artefactos exportados	Dificultad para probar inferencia tras clonar el proyecto.	Inclusión de modelos finales y metadatos ligeros en el repositorio.
Despliegue complejo	Problemas de instalación en otros equipos.	Uso de Docker Compose para levantar los servicios con una configuración común [5].
Trabajos de larga duración	Esperas en la API y dificultad para navegar.	Ejecución del análisis y entrenamiento en un worker de Celery.
Acceso a datos de otros usuarios	Exposición de datasets o experimentos ajenos.	Autenticación y comprobación de acceso en el backend.
Uso de datos clínicos reales	Riesgos legales y de privacidad.	Limitación a datasets anonimizados y uso académico/controlado.

Tabla A.6: Riesgos principales del proyecto y medidas adoptadas.

A.3. Estudio de viabilidad

Viabilidad económica

El estudio económico se plantea como una estimación académica del coste que tendría desarrollar el proyecto en un entorno profesional. No representa un presupuesto comercial real, sino una valoración razonada de los recursos utilizados.

Coste de personal

Se toma como referencia el perfil de un Ingeniero Informático Junior encargado de realizar tareas de investigación, desarrollo, pruebas y documentación. Para la estimación se considera un salario bruto anual de 20.000 € y un coste empresarial adicional del 32 % sobre el salario bruto, utilizado como supuesto de cálculo y no como una liquidación real de cotizaciones.

Concepto	Valor
Salario bruto anual de referencia	20.000 €
Salario mensual bruto, con extras prorrateadas	1.666,67 €
Coste empresarial adicional supuesto	533,33 €
Coste mensual total para la empresa	2.200,00 €
Horas laborables mensuales estimadas	160 h
Coste por hora	13,75 €/h

Tabla A.7: Coste mensual estimado del perfil de desarrollo.

El cálculo reparte el coste anual entre doce meses y utiliza 160 horas mensuales. Así se evita mezclar un salario dividido en catorce pagas con doce meses de horas laborables. Tanto el salario como el porcentaje adicional son supuestos para la estimación académica.

Concepto	Cálculo	Importe
Horas estimadas del proyecto	390 h	—
Coste por hora	13,75 €	—
Coste estimado de personal	$390 \times 13,75 \text{ €}$	5.362,50 €

Tabla A.8: Coste de personal imputado al proyecto.

Coste de hardware

El hardware se imputa mediante amortización lineal, considerando únicamente la parte proporcional al tiempo de uso durante el proyecto.

Equipo	Precio	Vida útil	Amort. mensual	Meses	Coste
Portátil de desarrollo	1.500 €	5 años	25,00 €	4,5	112,50 €
Monitor externo	220 €	5 años	3,67 €	4,5	16,50 €
Total hardware	—	—	—	—	129 €

Tabla A.9: Coste de hardware imputado con amortización lineal.

Coste de software

No se imputa un coste adicional de licencias al proyecto: se han utilizado bibliotecas libres, servicios en sus modalidades gratuitas y el sistema operativo disponible en el equipo de desarrollo.

Concepto	Coste
Sistema operativo utilizado durante el desarrollo	0 €
Visual Studio Code	0 €
Bibliotecas de Python y npm	0 €
GitHub	0 €
SonarCloud	0 €
Total software	0 €

Tabla A.10: Coste de software del proyecto.

Coste de infraestructura

El cálculo de la aplicación sigue ejecutándose en el equipo local y se publica mediante Cloudflare Tunnel. La landing se despliega por separado en Cloudflare Pages. La compra del dominio `eegscope.dev` ha supuesto un coste de 10 €.

Concepto	Coste
Hosting cloud	0 €
Dominio <code>eegscope.dev</code>	10 €
Servicios externos de pago	0 €
Total infraestructura	10 €

Tabla A.11: Coste de infraestructura del proyecto.

Resumen de costes

Categoría	Importe
Personal, estimación actualizada	5.362,50 €
Hardware	129 €
Software	0 €
Infraestructura	10 €
Total estimado	5.501,50 €

Tabla A.12: Resumen de costes del proyecto.

El total incluye la estimación actualizada de personal y la compra del dominio. Se mantiene el supuesto de 4,5 meses de uso del hardware para su amortización. El coste de personal representa una valoración académica del trabajo, no un desembolso salarial realizado.

Viabilidad legal

Licencia del software desarrollado

El código propio del proyecto se distribuye bajo licencia MIT, indicada en el fichero `LICENSE` de la raíz del repositorio. Se ha elegido esta licencia por ser permisiva y facilitar la reutilización del código [11].

La licencia MIT permite utilizar, modificar y distribuir el código, siempre que se mantengan el aviso de copyright y el texto de la licencia. Esta licencia no sustituye a las de las dependencias ni se aplica automáticamente al dataset utilizado.

Licencias de dependencias

La aplicación utiliza componentes con distintas licencias. La tabla recoge algunas de las dependencias principales y resume las condiciones que deben tenerse en cuenta al redistribuirlas. Estas condiciones también afectan a los componentes incluidos en las imágenes de los contenedores; utilizar Docker no modifica sus licencias.

Componente	Licencia	Condiciones principales
FastAPI y React	MIT	Conservar el aviso de copyright y el texto de la licencia.
scikit-learn	BSD de 3 cláusulas	Conservar los avisos y las condiciones. No utilizar el nombre de sus autores para promocionar productos derivados sin autorización.
TensorFlow	Apache 2.0	Incluir la licencia, identificar los cambios realizados y conservar los avisos aplicables, incluido NOTICE cuando exista.
PostgreSQL	PostgreSQL License	Conservar el aviso de copyright y el texto de la licencia.
Redis 8.2.7	AGPLv3, RSALv2 o SSPLv1	Cumplir las condiciones de una de las tres opciones ofrecidas para esta versión.

Tabla A.13: Licencias de los principales componentes utilizados.

FastAPI y React publican sus condiciones en los ficheros de licencia de sus repositorios oficiales¹. Las condiciones de scikit-learn, TensorFlow y PostgreSQL también pueden consultarse en sus fuentes oficiales².

Redis requiere una consideración aparte. El despliegue utiliza la imagen `redis:8.2.7-alpine`, y Redis 8 ofrece las alternativas AGPLv3, RSALv2 y SSPLv1 [15]. Sus condiciones son distintas, por lo que no se describe todo el despliegue como un conjunto de componentes con licencias permisivas.

¹<https://github.com/fastapi/fastapi/blob/master/LICENSE> y <https://github.com/facebook/react/blob/main/LICENSE>

²<https://github.com/scikit-learn/scikit-learn/blob/main/COPYING>, <https://github.com/tensorflow/tensorflow/blob/master/LICENSE> y <https://www.postgresql.org/about/licence/>

La tabla resume las licencias principales, pero no constituye un inventario completo de las dependencias indirectas ni de los paquetes incluidos en los contenedores. Para redistribuir el sistema deben conservarse los textos y avisos correspondientes a las versiones utilizadas.

Procedencia y licencia del dataset

El fichero `adhdata.csv` utilizado en los experimentos procede de la publicación *EEG Dataset for ADHD*, disponible en Kaggle bajo el usuario `danizo`³. Su descripción atribuye los registros originales a Ali Motie Nasrabadi, Armin Allahverdy, Mehdi Samavati y Mohammad Reza Mohammadi, y remite al conjunto publicado en IEEE DataPort con DOI `10.21227/rzfh-zn36`.

La descripción de Kaggle indica una licencia *Creative Commons Attribution*, sin especificar su versión. Esta licencia debe distinguirse de la licencia MIT del código propio. La reutilización de los datos requiere mantener la atribución a sus autores y respetar las condiciones de la publicación original.

El CSV no se incluye en el repositorio. Para reproducir los experimentos, debe obtenerse desde la fuente indicada y colocarse en la ruta descrita en las instrucciones de instalación.

Tratamiento de datos personales

El dataset utilizado en la fase experimental es un dataset público de EEG con identificadores de sujeto. La aplicación no necesita que los CSV incluyan nombre, DNI o historia clínica. Sin embargo, las cuentas sí almacenan el correo del usuario. Además, sustituir el nombre por un identificador no permite afirmar, por sí solo, que los datos EEG sean anónimos [18].

No obstante, se debe distinguir entre los distintos usos de la aplicación:

- **Datasets de entrenamiento:** los CSV multipaciente pueden almacenarse para reutilizarlos en experimentos posteriores. También se guardan metadatos como hash, nombre de fichero, tamaño, número de filas, columnas, sujetos, distribución de clases y canales detectados.
- **CSV de predicción:** los ficheros usados para inferencia individual se procesan durante la petición y no se almacenan como dataset persistente.

³<https://www.kaggle.com/datasets/danizo/eeg-dataset-for-adhd>

- **Cuentas de usuario:** se guardan el correo, el hash de la contraseña, el estado de la cuenta y la fecha de registro. Los endpoints privados comprueban la sesión y el acceso a los recursos.
- **Entorno de uso:** la aplicación está pensada para un contexto académico. La autenticación y el túnel no bastan para afirmar que se cumplen todas las obligaciones de protección de datos.

Por tanto, el sistema debe utilizarse únicamente con datasets anonimizados o pseudonimizados y dentro de un contexto académico/controlado. En caso de usar datos reales de sujetos o plantear un despliegue público, sería necesario revisar las obligaciones derivadas del RGPD y la LOPDGDD [18, 9], definir las condiciones del tratamiento y las políticas de conservación, y valorar medidas de seguridad adicionales.

Carácter no clínico

La aplicación tiene carácter académico y demostrativo. No constituye un producto sanitario ni debe utilizarse como herramienta de diagnóstico clínico. Sus resultados deben interpretarse únicamente como salida de un prototipo de clasificación desarrollado en el contexto de un Trabajo de Fin de Grado. Esta distinción es relevante respecto al marco regulatorio aplicable a los productos sanitarios en la Unión Europea [19].

Apéndice B

Especificación de Requisitos

B.1. Introducción

Propósito

En este apéndice se recogen los requisitos funcionales y no funcionales de la aplicación desarrollada. El objetivo es definir qué debe permitir hacer el sistema, qué restricciones debe cumplir y qué funcionalidades forman parte del alcance del trabajo.

La aplicación está orientada a la clasificación de señales EEG relacionadas con TDAH mediante modelos de Machine Learning y Deep Learning. Además de la inferencia sobre nuevos pacientes, el sistema permite cargar datasets, entrenar modelos, consultar experimentos y visualizar métricas de evaluación.

Alcance

El sistema desarrollado es una aplicación web de uso académico e investigador. Permite cargar datasets EEG multipaciente en formato CSV para analizarlos y utilizarlos en entrenamientos, así como cargar el CSV de un único paciente para obtener una clasificación ADHD/Control mediante un modelo previamente exportado.

La aplicación incluye también un histórico de experimentos, donde se almacenan los datasets utilizados, los parámetros de entrenamiento, las métricas obtenidas y los resultados por pliegue de validación. El sistema no tiene finalidad clínica y no debe utilizarse como herramienta de diagnóstico médico.

El acceso requiere una cuenta de usuario. Cada usuario consulta sus propios experimentos y modelos, y los datasets a los que tiene acceso. El análisis de datasets y los entrenamientos se ejecutan en segundo plano, por lo que no es necesario permanecer en la misma pestaña hasta que terminen.

B.2. Objetivos generales

Los objetivos principales del proyecto son los siguientes:

1. **Explorar datasets EEG multipaciente**, permitiendo cargar un fichero CSV y consultar información básica sobre el mismo, incluyendo número de filas, columnas, sujetos, distribución de clases y canales EEG disponibles.
2. **Entrenar modelos de clasificación ADHD/Control** desde la propia aplicación, pudiendo configurar parámetros relacionados con la segmentación de la señal EEG y con el modelo seleccionado.
3. **Realizar inferencia sobre nuevos pacientes**, cargando el CSV de un sujeto y eligiendo uno de los modelos entrenados por el usuario o uno de los modelos de referencia de Machine Learning y Deep Learning.
4. **Consultar la información de los modelos exportados**, mostrando sus metadatos, configuración, métricas de validación y figuras de evaluación asociadas.
5. **Comparar enfoques de Machine Learning y Deep Learning**, evaluando modelos clásicos y arquitecturas neuronales dentro del pipeline desarrollado.
6. **Mantener un histórico reproducible de experimentos**, almacenando los datasets utilizados, los parámetros de entrenamiento, las métricas obtenidas y los resultados por pliegue de validación.

B.3. Catálogo de requisitos

En las tablas, la prioridad se indica con 1 (alta), 2 (media) o 3 (baja). La exigencia distingue los requisitos esenciales (E) de los deseables (D).

Nro.	Funcionalidad	Prio.	Exig.
Módulo Modelo			
RFM01	Listar los modelos exportados disponibles.	1	E
RFM02	Consultar la información del modelo activo: tipo, configuración, métricas y metadatos.	1	E
RFM03	Visualizar las figuras de evaluación asociadas al modelo.	2	E
RFM04	Seleccionar un modelo propio o uno de referencia para inferencia.	1	E
Módulo Dataset			
RFD01	Cargar un CSV con datos EEG multipaciente como dataset de entrenamiento.	1	E
RFD02	Validar el formato del CSV: canales EEG, columna ID y columna Class.	1	E
RFD03	Calcular y mostrar estadísticas básicas del dataset.	1	E
RFD04	Guardar datasets de entrenamiento para reutilizarlos en experimentos posteriores.	2	E
RFD05	Filtrar la lista de pacientes mostrada por clase.	3	D
Módulo Entrenamiento			
RFE01	Configurar los parámetros de procesado EEG.	1	E
RFE02	Elegir el tipo de modelo, el modelo concreto y sus hiperparámetros.	1	E
RFE03	Configurar los parámetros del entrenamiento en modelos de Deep Learning.	2	E
RFE04	Ejecutar entrenamiento con validación <i>cross-subject</i> .	1	E
RFE05	Mostrar el estado del entrenamiento sin bloquear la navegación.	2	E
RFE06	Mostrar métricas agregadas, matriz de confusión, informe de clasificación y resultados por paciente.	1	E
RFE07	Mostrar importancia de características en los modelos de Machine Learning cuando esté disponible.	3	D

Tabla B.1: Catálogo de requisitos funcionales de los módulos de modelo, dataset y entrenamiento.

Nro.	Funcionalidad	Prio.	Exig.
Módulo Experimentos			
RFX01	Guardar los resultados y parámetros de los entrenamientos completados.	1	E
RFX02	Evitar duplicados de datasets mediante hash SHA-256.	2	E
RFX03	Mostrar un listado de los experimentos del usuario.	1	E
RFX04	Consultar el detalle completo de un experimento, incluyendo configuración y resultados por pliegue.	1	E
Módulo Predicción			
RFP01	Cargar el CSV de un paciente individual para predicción.	1	E
RFP02	Validar el CSV del paciente contra el modelo seleccionado.	1	E
RFP03	Predecir la clase TDAH/Control agregando los votos por epoch.	1	E
RFP04	Mostrar la distribución de epochs por clase, la decisión final y el score de confianza.	2	E
Módulo Interfaz			
RFI01	Cambiar el idioma de la interfaz entre español e inglés.	3	D

Tabla B.2: Catálogo de requisitos funcionales de los módulos de experimentos, predicción e interfaz.

Nro.	Funcionalidad	Prio.	Exig.
Usuarios y trabajos en segundo plano			
RFU01	Registrar una cuenta con correo único y contraseña.	1	E
RFU02	Iniciar sesión, consultar el perfil y cerrar sesión.	1	E
RFU03	Comprobar el acceso del usuario a datasets, experimentos, modelos y tareas.	1	E
RFD06	Ejecutar el análisis del dataset en segundo plano y mostrar su estado.	1	E
RFE08	Ejecutar los entrenamientos ML y DL en segundo plano.	1	E
RFE09	Exportar y registrar el modelo final para utilizarlo en inferencia.	1	E
RFM05	Mostrar el mejor modelo propio disponible y permitir usarlo en inferencia.	2	E

Tabla B.3: Requisitos incorporados en la ampliación de la aplicación.

B.4. Casos de uso principales

Diagrama de casos de uso

La Figura B.1 recoge las interacciones principales de la aplicación. Se distingue entre el visitante, que puede registrarse e iniciar sesión, y el usuario autenticado, que accede a los datasets, entrenamientos, modelos y predicciones.

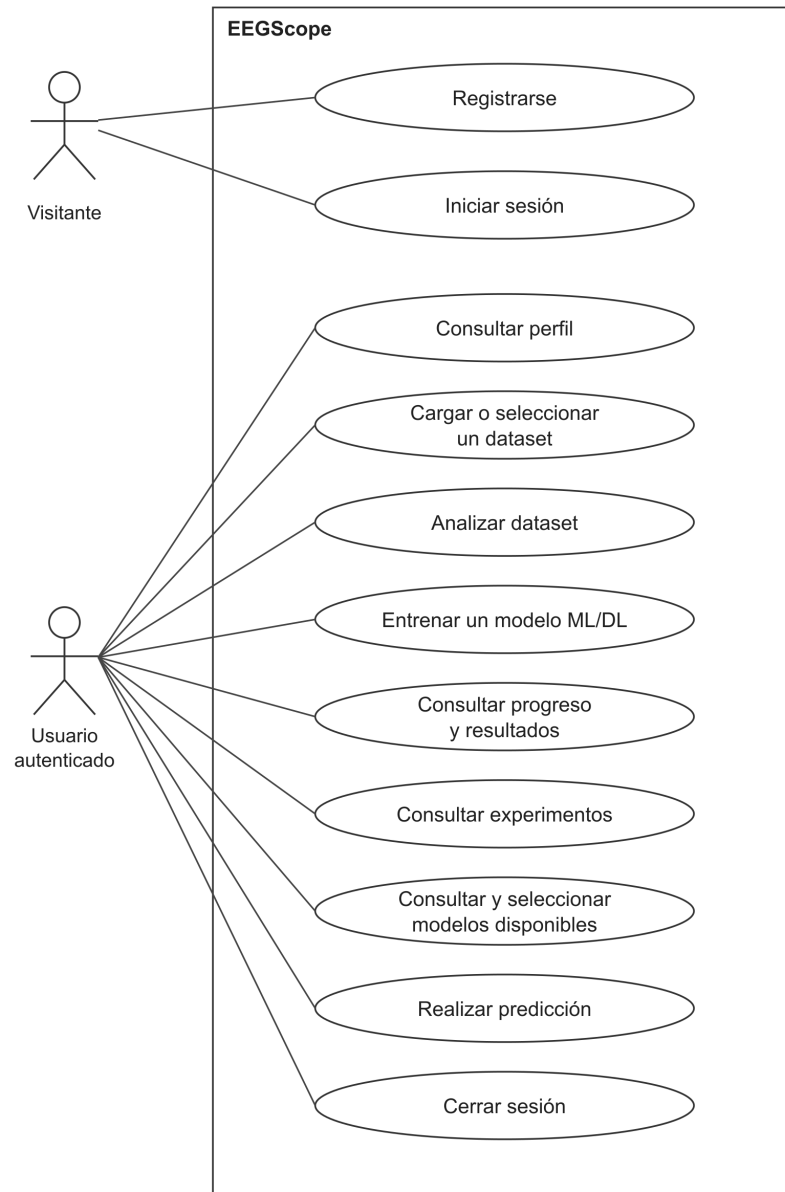


Figura B.1: Casos de uso del visitante y del usuario autenticado.

Descripción de los casos de uso

Salvo el registro y el inicio de sesión, los casos relacionados con datos y modelos requieren una sesión válida. Las consultas se limitan a los recursos accesibles para ese usuario.

CU-01	Consultar información del modelo de inferencia
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFM01, RFM02, RFM03, RFM04
Descripción	El usuario consulta la información del modelo que se va a usar para inferencia, junto con sus métricas y figuras de evaluación.
Precondición	Hay al menos un modelo exportado en el sistema.
Acciones	1. El usuario abre la pestaña “Modelo”. 2. Elige en el desplegable un modelo propio o uno de referencia. 3. Aparecen los metadatos, las métricas y las figuras que tenga disponibles ese modelo.
Postcondición	El usuario tiene a la vista la información del modelo seleccionado.
Excepciones	Si no hay ningún modelo exportado, se muestra un aviso indicando que no hay nada que mostrar.
Importancia	Alta

Tabla B.4: CU-01 Consultar información del modelo de inferencia.

CU-02	Cargar un dataset de entrenamiento
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFD01, RFD04
Descripción	El usuario sube un CSV multipaciente o selecciona uno que ya esté guardado para usarlo más adelante en un entrenamiento.
Precondición	El usuario dispone de un CSV con los 19 canales 10-20, una columna ID y una columna Class .
Acciones	1. El usuario abre la pestaña “Dataset entrenamiento”. 2. Pulsa el cuadro de selección de CSV y elige el archivo, o bien selecciona uno de los datasets guardados. 3. Si es un dataset nuevo, queda almacenado para poder reutilizarlo en futuros entrenamientos.
Postcondición	El dataset está cargado y listo para analizar o entrenar.
Excepciones	Si el archivo no es un CSV válido, no se carga y se avisa al usuario.
Importancia	Alta

Tabla B.5: CU-02 Cargar un dataset de entrenamiento.

CU-03	Analizar el dataset y filtrar pacientes
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFD02, RFD03, RFD05, RFD06
Descripción	El usuario lanza el análisis del dataset cargado para ver sus estadísticas y filtrar los pacientes por clase.
Precondición	Hay un dataset cargado (CU-02).
Acciones	1. El usuario pulsa el botón “Analizar dataset”. 2. El sistema encola el análisis y muestra su estado mientras el worker calcula las estadísticas. 3. Aparece el resumen junto con la tabla de pacientes. 4. El usuario puede filtrar la tabla por clase.
Postcondición	El usuario conoce la estructura del dataset y puede decidir si lo usa para entrenar.
Excepciones	Si faltan columnas obligatorias o no se reconoce alguna etiqueta de clase, se muestra un mensaje describiendo el problema.
Importancia	Alta

Tabla B.6: CU-03 Analizar el dataset y filtrar pacientes.

CU-04	Configurar y ejecutar un entrenamiento
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFE01–RFE09, RFX01, RFX02
Descripción	El usuario ajusta los parámetros del entrenamiento y lo lanza. El sistema evalúa el modelo con validación cruzada <i>cross-subject</i> y guarda el resultado.
Precondición	Hay un dataset cargado (CU-02) y analizado (CU-03).
Acciones	1. El usuario configura la ventana, el solape, la frecuencia, las características y <code>nperseg</code> cuando aplica. 2. Selecciona un modelo de Machine Learning o Deep Learning y sus hiperparámetros. 3. En Deep Learning, ajusta el tamaño de batch, las épocas, el <i>learning rate</i> y la paciencia. 4. Pulsa el botón “Entrenar”. Se encola el trabajo y puede seguir navegando entre pestañas. 5. El sistema prepara las epochs y aplica <code>StratifiedGroupKFold</code> para que ningún paciente aparezca en train y test a la vez. 6. Al terminar, aparecen las métricas agregadas, la matriz de confusión, los resultados por paciente y, en el caso de ML, la importancia de las características. 7. Se guardan el experimento y un modelo final para inferencia, indicando si alguna de estas operaciones falla.
Postcondición	El usuario ve los resultados y el experimento queda registrado para consultas posteriores.
Excepciones	Si los datos o parámetros no son válidos, se informa del fallo. Si falla el guardado del modelo, las métricas pueden conservarse sin disponer del artefacto para inferencia.
Importancia	Alta

Tabla B.7: CU-04 Configurar y ejecutar un entrenamiento.

CU-05	Consultar el histórico de experimentos
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFX03, RFX04
Descripción	El usuario revisa los entrenamientos guardados y abre el detalle de uno concreto para ver su configuración y sus métricas.
Precondición	Hay al menos un experimento guardado.
Acciones	1. El usuario abre la pestaña “Experimentos”. 2. Aparece la tabla con sus experimentos y el resumen del mejor modelo propio disponible. 3. El usuario selecciona uno de la lista. 4. Se muestra el detalle del dataset, los parámetros y las métricas. Si tiene un modelo exportado, puede seleccionarlo para inferencia.
Postcondición	El usuario puede comparar varios entrenamientos y decidir cuál ha funcionado mejor.
Excepciones	Si no hay experimentos guardados, la pestaña se muestra vacía con un mensaje informativo.
Importancia	Media

Tabla B.8: CU-05 Consultar el histórico de experimentos.

CU-06	Validar el CSV de un paciente
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFP01, RFP02
Descripción	El usuario sube el CSV de un paciente y el sistema comprueba que es compatible con el modelo seleccionado antes de permitir lanzar la predicción.
Precondición	Hay un modelo de inferencia seleccionado.
Acciones	1. El usuario abre la pestaña “Predicción”. 2. Sube el CSV del paciente desde el cuadro de selección. 3. El sistema comprueba que están los canales esperados por el modelo seleccionado y devuelve información básica del fichero. 4. Aparece la información del archivo y la lista de canales esperados.
Postcondición	El usuario sabe si el archivo es apto para predecir.
Excepciones	Si el CSV no es compatible con el modelo, se muestra un mensaje indicando qué falta.
Importancia	Alta

Tabla B.9: CU-06 Validar el CSV de un paciente.

CU-07	Ejecutar predicción y revisar resultado
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFP03, RFP04
Descripción	El usuario lanza la predicción sobre el CSV ya validado y consulta la clase final, la confianza y la distribución de las ventanas analizadas.
Precondición	El CSV ha sido validado correctamente (CU-06).
Acciones	1. El usuario pulsa el botón “Ejecutar predicción”. 2. El sistema segmenta la señal en ventanas y pasa cada una por el modelo. 3. Los votos de cada ventana se agregan por mayoría para obtener la clase final. 4. Aparece el resultado: clase (ADHD o Control), confianza media, número de ventanas analizadas y una gráfica con la distribución por clase.
Postcondición	El usuario obtiene la clasificación del paciente con suficiente información para interpretarla.
Excepciones	Si el CSV no permite generar ninguna ventana, por ejemplo por ser demasiado corto, se muestra un error explicando el problema.
Importancia	Alta

Tabla B.10: CU-07 Ejecutar predicción y revisar resultado.

CU-08	Cambiar el modelo activo para inferencia
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFM04
Descripción	El usuario cambia el modelo que se usa para predecir entre los modelos propios y los de referencia disponibles.
Precondición	Hay otro modelo accesible que se puede seleccionar.
Acciones	1. El usuario abre el desplegable de modelo, visible tanto en la pestaña “Modelo” como en la pestaña “Predicción”. 2. Elige otro modelo entre los disponibles. 3. El cambio queda registrado y el modelo seleccionado pasa a ser el usado en las siguientes predicciones.
Postcondición	El modelo activo cambia y se usa a partir de ese momento.
Excepciones	Si no hay modelo del tipo elegido, no aparece como opción seleccionable.
Importancia	Media

Tabla B.11: CU-08 Cambiar el modelo activo para inferencia.

CU-09	Cambiar el idioma de la interfaz
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFI01
Descripción	El usuario cambia el idioma de la aplicación entre español e inglés.
Precondición	La aplicación está abierta.
Acciones	1. El usuario abre el selector de idioma de la cabecera. 2. Elige el idioma deseado. 3. Los textos de la interfaz se actualizan sin perder el estado actual.
Postcondición	La interfaz queda traducida al idioma elegido.
Excepciones	Ninguna relevante.
Importancia	Baja

Tabla B.12: CU-09 Cambiar el idioma de la interfaz.

CU-10	Registrar una cuenta e iniciar sesión
Versión	1.0
Autor	Adrián Jiménez García
Requisitos asociados	RFU01, RFU02, RFU03
Descripción	El usuario accede a la aplicación con su correo y contraseña.
Precondición	La aplicación está disponible.
Acciones	1. Si no tiene cuenta, abre el registro e introduce correo y contraseña. 2. El sistema crea la cuenta si el correo no existe y los datos son válidos. 3. El usuario inicia sesión y accede a sus recursos. 4. Desde el perfil puede consultar los datos de su cuenta y cerrar sesión.
Postcondición	El usuario puede utilizar las funciones privadas mientras la sesión sea válida.
Excepciones	Se informa si el correo ya existe, las credenciales son incorrectas o la sesión ha caducado.
Importancia	Alta

Tabla B.13: CU-10 Registrar una cuenta e iniciar sesión.

B.5. Requisitos no funcionales

Además de las funcionalidades anteriores, el sistema debe cumplir una serie de requisitos no funcionales:

- **RNF01. Portabilidad:** la aplicación debe poder ejecutarse mediante Docker Compose, reduciendo la configuración manual del entorno.
- **RNF02. Mantenibilidad:** el código debe estar separado por responsabilidades entre frontend, backend, servicios, modelos y scripts de investigación.
- **RNF03. Reproducibilidad:** los experimentos deben guardar dataset, parámetros y métricas para poder revisar los resultados posteriormente.

- **RNF04. Validación metodológica:** la evaluación de los modelos debe evitar mezclar pacientes entre entrenamiento y prueba mediante validación *cross-subject*.
- **RNF05. Seguridad básica:** los endpoints privados deben validar la sesión y el acceso a los recursos. Las contraseñas se almacenan mediante hash y los secretos quedan fuera del repositorio.
- **RNF06. Carácter no clínico:** la aplicación no debe presentarse como herramienta de diagnóstico médico, sino como prototipo académico.
- **RNF07. Ejecución asíncrona:** el análisis y el entrenamiento no deben mantener abierta la petición HTTP durante todo el cálculo ni bloquear la navegación.

Apéndice C

Especificación de diseño

C.1. Introducción

En este anexo se describe el diseño del software desde tres perspectivas: diseño de datos, diseño arquitectónico y diseño procedimental.

El sistema desarrollado es una aplicación web orientada a un uso académico y de investigación. Permite cargar datasets EEG multipaciente para entrenar modelos de Machine Learning y Deep Learning, así como cargar el CSV de un único paciente para obtener una clasificación ADHD/Control mediante inferencia. Además, la aplicación permite consultar el histórico de experimentos realizados y las métricas de los modelos exportados.

Los diagramas incluidos en este anexo se utilizan como apoyo visual para explicar la estructura de datos, la arquitectura de la aplicación y los flujos principales del sistema.

C.2. Diseño de datos

El sistema utiliza una base de datos relacional PostgreSQL para mantener un histórico de los entrenamientos lanzados desde la interfaz. El esquema incluye usuarios, datasets, permisos de acceso, experimentos, resultados por pliegue, modelos entrenados y trabajos en segundo plano.

- **datasets 1:N experiments.** Un mismo dataset puede reutilizarse en varios experimentos.

- **experiments** 1:N **experiment_folds**. Cada experimento genera varios pliegues de validación.
- **users** N:M **datasets**, mediante **dataset_accesses**. Un CSV puede estar deduplicado sin que todos los usuarios puedan acceder a él.
- **users** 1:N **experiments**. Cada nuevo experimento pertenece al usuario que lo ejecuta.
- **experiments** 1:0..1 **trained_models**. Un experimento puede tener un modelo final exportado.
- **users** 1:N **background_jobs**. Cada trabajo encolado tiene un propietario.

La entidad **datasets** almacena información del CSV utilizado, incluyendo su hash SHA-256, nombre, ruta de almacenamiento, tamaño, dimensiones, distribución de clases y canales EEG detectados. La entidad **experiments** guarda cada entrenamiento ejecutado junto con su configuración y métricas globales. Por último, **experiment_folds** almacena las métricas obtenidas en cada pliegue de la validación cruzada *cross-subject*.

SQLAlchemy define borrado en cascada para los experimentos asociados a un dataset, y para los pliegues y el registro del modelo asociados a un experimento. Esto no equivale a borrar automáticamente los ficheros del disco ni implica que exista una operación de borrado disponible en la interfaz.

Los ficheros CSV y los modelos exportados se guardan fuera de PostgreSQL. La base de datos conserva sus rutas y metadatos. Los modelos entrenados heredan el acceso del experimento al que pertenecen.

En el diagrama se muestran los atributos principales. PK indica la clave primaria, FK una clave foránea y UQ una restricción de unicidad.

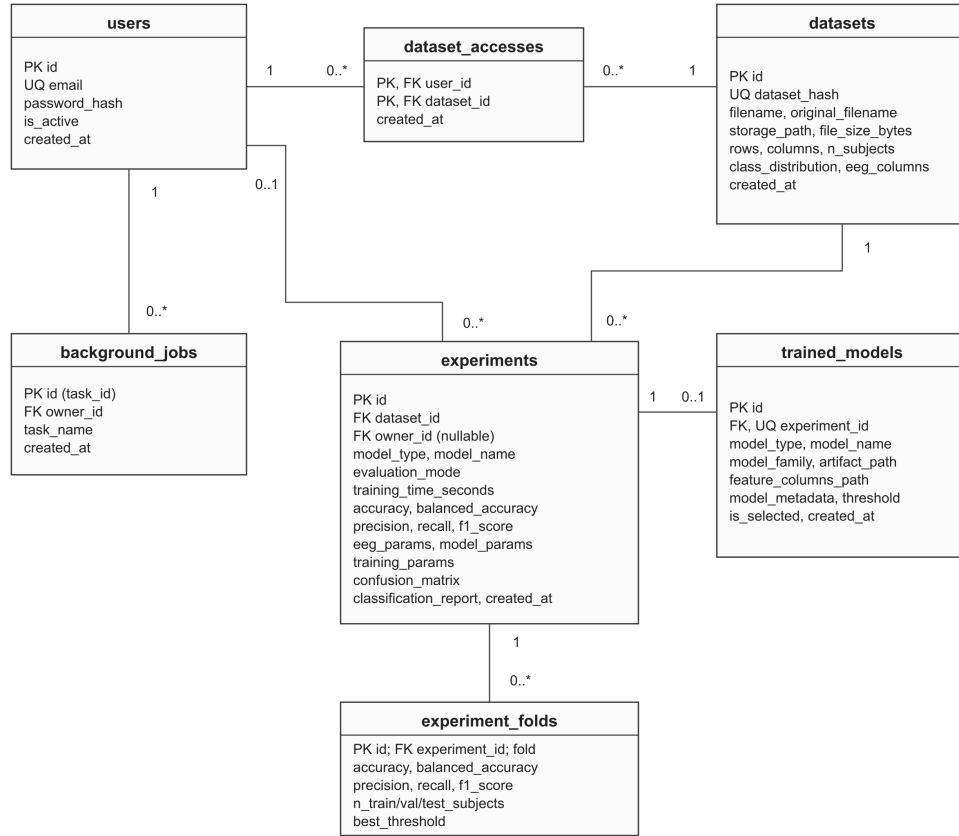


Figura C.1: Esquema de usuarios, acceso a datasets, experimentos, modelos entrenados y trabajos en segundo plano.

Diccionario de datos

A continuación se resumen las tablas principales de la base de datos. Se mantiene el mismo formato para todas ellas: atributo, tipo, restricciones y descripción.

Tabla datasets

La tabla **datasets** almacena los datasets subidos desde la interfaz para su reutilización en entrenamientos posteriores.

Atributo	Tipo	Restricciones	Descripción
id	INTEGER	PK	Identificador del dataset.
dataset_hash	VARCHAR(64)	UNIQUE, NOT NULL	Hash SHA-256 para evitar duplicados.
filename	VARCHAR(255)	NOT NULL	Nombre interno del fichero.
original_filename	VARCHAR(255)	NULL	Nombre original subido.
storage_path	VARCHAR(500)	NULL	Ruta del CSV almacenado.
file_size_bytes	INTEGER	NULL	Tamaño del fichero.
rows	INTEGER	NOT NULL	Número de filas.
columns	INTEGER	NOT NULL	Número de columnas.
n_subjects	INTEGER	NOT NULL	Número de sujetos.
class_distribution	JSON	NOT NULL	Distribución de clases.
eeg_columns	JSON	NOT NULL	Canales EEG detectados.
created_at	DATETIME	NOT NULL	Fecha de registro.

Tabla C.1: Diccionario de datos de la tabla **datasets**.

Tabla experiments

La tabla `experiments` guarda cada entrenamiento ejecutado desde la interfaz, junto con sus parámetros, métricas agregadas e informes asociados.

Atributo	Tipo	Restricciones	Descripción
<code>id</code>	INTEGER	PK	Identificador del experimento.
<code>created_at</code>	DATETIME	NOT NULL	Fecha de creación.
<code>dataset_id</code>	INTEGER	FK, NOT NULL	Dataset utilizado.
<code>owner_id</code>	INTEGER	FK, NULL	Usuario propietario; admite registros anteriores sin usuario.
<code>model_type</code>	VARCHAR(20)	NOT NULL	Tipo de modelo: ML o DL.
<code>model_name</code>	VARCHAR(80)	NOT NULL	Modelo entrenado.
<code>evaluation_mode</code>	VARCHAR(255)	NOT NULL	Estrategia de evaluación.
<code>training_time_seconds</code>	FLOAT	NOT NULL	Tiempo de entrenamiento.
<code>accuracy</code>	FLOAT	NOT NULL	Exactitud global.
<code>balanced_accuracy</code>	FLOAT	NOT NULL	Exactitud balanceada.
<code>precision</code>	FLOAT	NOT NULL	Precisión global.
<code>recall</code>	FLOAT	NOT NULL	Recall ponderado global.
<code>f1_score</code>	FLOAT	NOT NULL	F1-score global.
<code>eeg_params</code>	JSON	NOT NULL	Parámetros EEG.
<code>model_params</code>	JSON	NOT NULL	Hiperparámetros.
<code>training_params</code>	JSON	NOT NULL	Parámetros de entrenamiento.
<code>confusion_matrix</code>	JSON	NOT NULL	Matriz de confusión.
<code>classification_report</code>	JSON	NOT NULL	Informe de clasificación.

Tabla C.2: Diccionario de datos de la tabla `experiments`.

Tabla `experiment_folds`

La tabla `experiment_folds` guarda las métricas obtenidas en cada pliegue de la validación cruzada *cross-subject* asociada a un experimento concreto.

Atributo	Tipo	Restricciones	Descripción
<code>id</code>	INTEGER	PK	Identificador del pliegue.
<code>experiment_id</code>	INTEGER	FK, NOT NULL	Experimento asociado.
<code>fold</code>	INTEGER	NOT NULL	Número de pliegue.
<code>accuracy</code>	FLOAT	NOT NULL	Exactitud del pliegue.
<code>balanced_accuracy</code>	FLOAT	NOT NULL	Exactitud balanceada.
<code>precision</code>	FLOAT	NOT NULL	Precisión del pliegue.
<code>recall</code>	FLOAT	NOT NULL	Recall ponderado del pliegue.
<code>f1_score</code>	FLOAT	NOT NULL	F1-score del pliegue.
<code>n_train_subjects</code>	INTEGER	NULL	Sujetos de entrenamiento.
<code>n_val_subjects</code>	INTEGER	NULL	Sujetos de validación.
<code>n_test_subjects</code>	INTEGER	NULL	Sujetos de prueba.
<code>best_threshold</code>	FLOAT	NULL	Umbral óptimo, si aplica.

Tabla C.3: Diccionario de datos de la tabla `experiment_folds`.

Usuarios y acceso a datasets

La tabla `users` guarda el identificador, el correo único, el hash de la contraseña, el estado de la cuenta y la fecha de creación. No se almacena la contraseña en texto plano. La tabla `dataset_accesses` relaciona usuarios y datasets mediante una clave primaria compuesta.

Atributo	Tipo	Restricciones	Descripción
<code>id</code>	INTEGER	PK	Identificador del usuario.
<code>email</code>	VARCHAR(320)	UNIQUE, NOT NULL	Correo de acceso.
<code>password_hash</code>	VARCHAR(255)	NOT NULL	Hash de la contraseña.
<code>is_active</code>	BOOLEAN	NOT NULL	Estado de la cuenta.
<code>created_at</code>	DATETIME	NOT NULL	Fecha de registro.

Tabla C.4: Diccionario de datos de la tabla `users`.

Tabla `dataset_accesses`

La tabla `dataset_accesses` relaciona cada usuario con los datasets que puede utilizar. Los campos `user_id` y `dataset_id` forman una clave

primaria compuesta, por lo que una misma relación usuario-dataset no puede aparecer dos veces.

El fichero puede reutilizarse sin duplicarlo en disco, mientras que el acceso se registra por separado para cada usuario.

Atributo	Tipo	Restricciones	Descripción
<code>user_id</code>	INTEGER	PK compuesta, FK, NOT NULL	Usuario con acceso. Referencia a <code>users.id</code> .
<code>dataset_id</code>	INTEGER	PK compuesta, FK, NOT NULL	Dataset accesible. Referencia a <code>datasets.id</code> .
<code>created_at</code>	DATETIME	NOT NULL	Fecha de registro del acceso.

Tabla C.5: Diccionario de datos de la tabla `dataset_accesses`.

Las dos claves foráneas tienen borrado en cascada: al eliminar el usuario o el dataset, se elimina la relación de acceso correspondiente.

Tabla `trained_models`

La tabla `trained_models` registra los modelos exportados y los relaciona con sus experimentos. Conserva la ubicación del fichero y los metadatos necesarios para utilizarlo, pero no almacena los pesos del modelo dentro de PostgreSQL.

El campo `experiment_id` es único, de modo que cada experimento puede tener como máximo un registro de modelo exportado.

Atributo	Tipo	Restricciones	Descripción
<code>id</code>	INTEGER	PK	Identificador del modelo.
<code>created_at</code>	DATETIME	NOT NULL	Fecha de registro.
<code>experiment_id</code>	INTEGER	FK, UNIQUE, NOT NULL	Experimento asociado. Referencia a <code>experiments.id</code> .
<code>model_type</code>	VARCHAR(20)	NOT NULL	Tipo de modelo: ML o DL.
<code>model_name</code>	VARCHAR(80)	NOT NULL	Nombre del modelo.
<code>model_family</code>	VARCHAR(40)	NOT NULL	Familia del modelo registrado.
<code>artifact_path</code>	VARCHAR(500)	NOT NULL	Ruta del fichero del modelo.
<code>feature_columns_path</code>	VARCHAR(500)	NULL	Ruta del fichero con las columnas de características, si aplica.
<code>n_features</code>	INTEGER	NULL	Número de características de entrada, si aplica.
<code>n_epochs_training</code>	INTEGER	NOT NULL	Número de ventanas EEG utilizadas para entrenar el modelo exportado.
<code>n_subjects_training</code>	INTEGER	NOT NULL	Número de sujetos utilizados en ese entrenamiento.
<code>file_size_bytes</code>	INTEGER	NULL	Tamaño del fichero en bytes.
<code>threshold</code>	FLOAT	NULL	Umbral de decisión, si aplica.
<code>model_metadata</code>	JSON	NOT NULL	Metadatos adicionales del modelo.
<code>is_selected</code>	BOOLEAN	NOT NULL	Indicador de selección del registro.

Tabla C.6: Diccionario de datos de la tabla `trained_models`.

En `n_epochs_training`, el término *epochs* se refiere a los segmentos o ventanas EEG, no al número de épocas de optimización de una red neuronal.

El acceso al modelo se comprueba a través del experimento al que pertenece, por lo que esta tabla no incorpora un campo propio de usuario propietario.

Tabla `background_jobs`

La tabla `background_jobs` relaciona los trabajos de Celery con sus propietarios. Permite comprobar si el usuario que consulta una tarea tiene permiso para acceder a ella.

Atributo	Tipo	Restricciones	Descripción
id	VARCHAR(255)	PK	Identificador de la tarea de Celery.
owner_id	INTEGER	FK, NOT NULL	Usuario propietario. Referencia a <code>users.id</code> .
task_name	VARCHAR(100)	NOT NULL	Nombre de la tarea.
created_at	DATETIME	NOT NULL	Fecha de registro.

Tabla C.7: Diccionario de datos de la tabla `background_jobs`.

La clave foránea `owner_id` tiene borrado en cascada. El estado y el resultado temporal del trabajo se consultan mediante Celery y Redis; no se almacenan en esta tabla.

C.3. Diseño arquitectónico

Visión general

El sistema se divide en los siguientes componentes principales:

- **Base de datos PostgreSQL**, donde se guardan usuarios, accesos a datasets, experimentos, métricas y registros de modelos y tareas.
- **Frontend** en React, TypeScript y Vite, con páginas de acceso y perfil y cinco vistas principales: Modelo, Dataset, Entrenamiento, Experimentos y Predicción.
- **Backend en FastAPI**, que expone la API REST utilizada por el frontend, gestiona la validación, el entrenamiento, la inferencia y la comunicación con la base de datos.
- **Scripts de investigación**, encargados de entrenar y exportar los modelos finales que el backend utiliza posteriormente para la inferencia.
- **Celery y Redis**, encargados de la cola y la ejecución de los análisis y entrenamientos en segundo plano.

Los servicios de la aplicación se despliegan juntos mediante Docker Compose. Los entrenamientos de investigación se lanzan por separado, aunque el backend reutiliza funciones de procesamiento de `scripts/`.

La figura muestra los componentes de la aplicación en ejecución. Las flechas indican qué componente utiliza cada servicio o recurso, no una transferencia de datos exclusivamente en un sentido. Los ficheros agrupan datasets, modelos entrenados y modelos de referencia; los pesos no se almacenan en PostgreSQL. La publicación mediante Caddy y Cloudflare se trata en el despliegue.

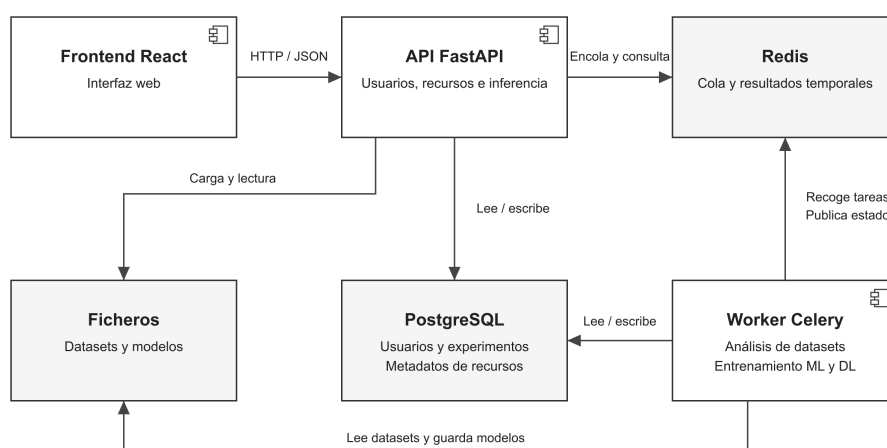


Figura C.2: Componentes de la aplicación: API, frontend, worker, Redis, PostgreSQL y ficheros utilizados.

Arquitectura del backend

El backend se ha reorganizado por dominios funcionales. Dentro de cada dominio se separan las responsabilidades de la API, los servicios y la persistencia:

- **auth**: registro, autenticación y comprobación del usuario actual.
- **datasets y training**: carga y análisis de datos, configuración, evaluación y exportación del modelo final.
- **experiments y model_registry**: histórico de experimentos y catálogo de modelos accesibles.
- **inference**: lectura del CSV, carga del predictor y clasificación.
- **worker**: configuración de Celery y control del propietario de los trabajos.

- **api, core, db y modeling:** elementos comunes de la API, configuración, conexión y modelos SQLAlchemy, y fábricas de ML/DL.

Esta separación permite que los endpoints se mantengan simples y que la lógica de negocio quede centralizada en los servicios, facilitando el mantenimiento y las pruebas del backend.

El diagrama resume los dominios y los módulos de apoyo, sin representar cada importación entre ficheros.

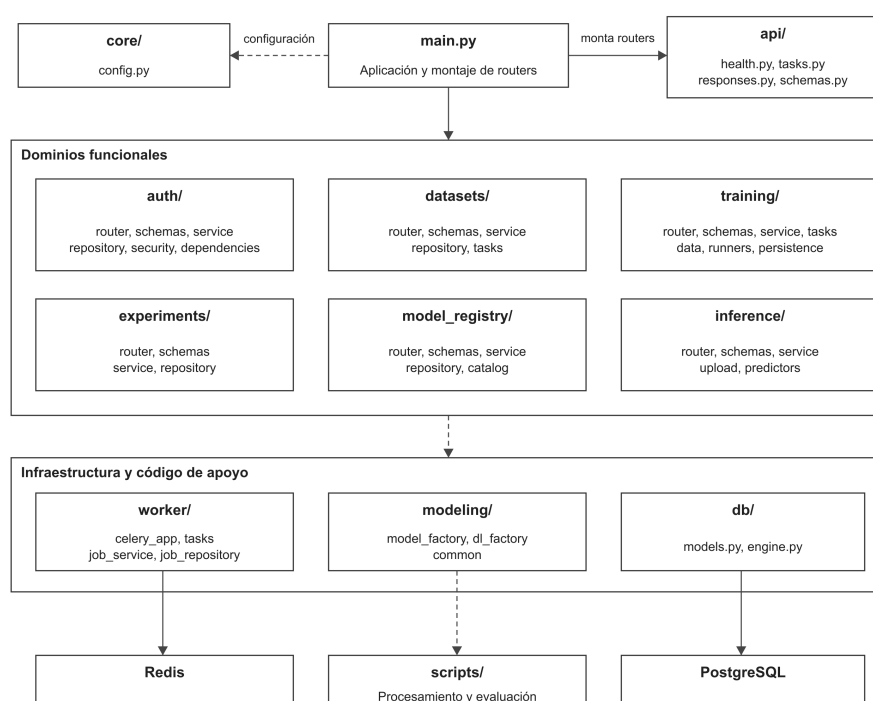


Figura C.3: Organización del backend por dominios funcionales y módulos compartidos.

Arquitectura del frontend

El frontend se ha desarrollado como una SPA (*Single Page Application*) en React, TypeScript y Vite. La interfaz se organiza en cinco pestañas que estructuran el flujo principal de uso:

- **Modelo:** muestra la información y métricas del modelo activo junto con sus figuras de evaluación.

- **Dataset:** permite cargar un CSV multipaciente y consultar sus estadísticas exploratorias.
- **Entrenamiento:** permite configurar parámetros EEG, de modelo y de entrenamiento, y lanzar la validación cruzada *cross-subject*.
- **Experimentos:** permite consultar el histórico de entrenamientos guardados en la base de datos.
- **Predicción:** permite subir el CSV de un paciente y obtener su clasificación ADHD/Control.

Internamente la SPA se compone de los siguientes módulos:

- **Vistas:** componentes principales por pestaña, como `ModelView`, `DatasetView`, `TrainingView`, `ExperimentsView` y `PredictionView`.
- **Componentes reutilizables:** elementos compartidos como `ModelSelector`, `Tabs` y paneles internos de entrenamiento.
- **Hooks personalizados:** gestionan el estado de cada flujo. Entre ellos están `useInferenceController`, `useTrainingDataset` y `useTrainingTask`.
- **Cliente HTTP:** cada funcionalidad tiene su `api.ts`, que utiliza el cliente común de `shared/api/`.
- **Visualización:** `Recharts` se utiliza para representar métricas y distribuciones de predicción.

Esta organización mantiene cada pestaña separada en cuanto a presentación, pero comparte el estado relevante mediante hooks para evitar duplicar peticiones al backend.

Los componentes se agrupan en `features/`, los elementos compartidos en `shared/` y la navegación en `app/`. `React Router` separa login, registro, perfil y aplicación. Las rutas privadas comprueban la sesión antes de mostrar su contenido; el backend vuelve a comprobar el acceso en cada petición.

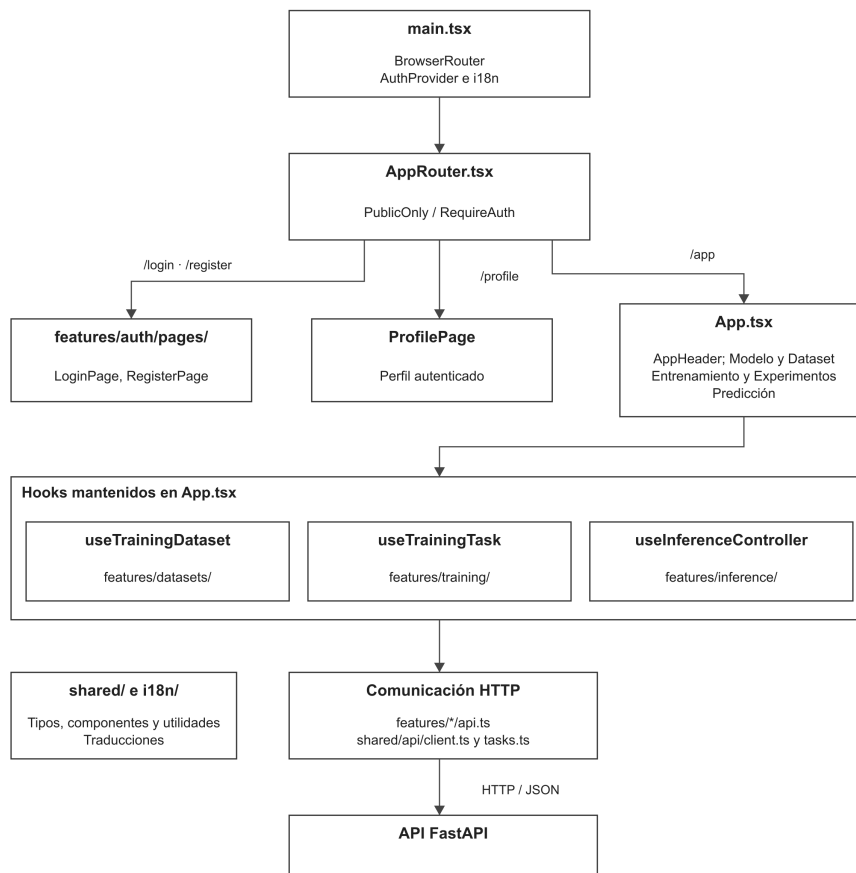


Figura C.4: Rutas, vistas, hooks y comunicación con la API en el frontend TypeScript.

Diagrama de despliegue

La configuración local utiliza Docker Compose, levantando cinco servicios contenedorizados:

- **db**: contenedor con `postgres:16-alpine`. Expone el puerto 5432, usa el volumen `postgres_data` e incluye un `healthcheck` con `pg_isready`.
- **backend**: contenedor con FastAPI servido por Uvicorn en el puerto 8000. Antes de aceptar peticiones ejecuta `alembic upgrade head` para mantener actualizado el esquema de la base de datos.
- **frontend**: contenedor que construye la aplicación con Vite y sirve los ficheros estáticos mediante Nginx no privilegiado. Internamente

escucha en el puerto 8080 y Docker Compose lo expone en el puerto 5173.

- **redis**: almacena la cola de tareas y sus resultados temporales. Utiliza un volumen persistente y no publica su puerto al anfitrión.
- **worker**: ejecuta las tareas de Celery con concurrencia uno. Comparte con el backend los CSV y los modelos entrenados mediante volúmenes.

Las credenciales de la base de datos y el secreto JWT se inyectan mediante un fichero `.env` que no se versiona en el repositorio. Además, los CSV de entrenamiento subidos desde la interfaz se guardan en un volumen persistente, lo que permite reutilizarlos en ejecuciones posteriores.

Para la publicación se añade `compose.prod.yml`, con Caddy como proxy inverso y un conector de Cloudflare Tunnel. El tráfico de <https://app.eegscope.dev> llega a Caddy: las rutas `/api/` se envían al backend y el resto al frontend. Los contenedores siguen ejecutándose en el equipo anfitrión, por lo que su disponibilidad depende de que esté encendido. La landing se publica por separado en Cloudflare Pages.

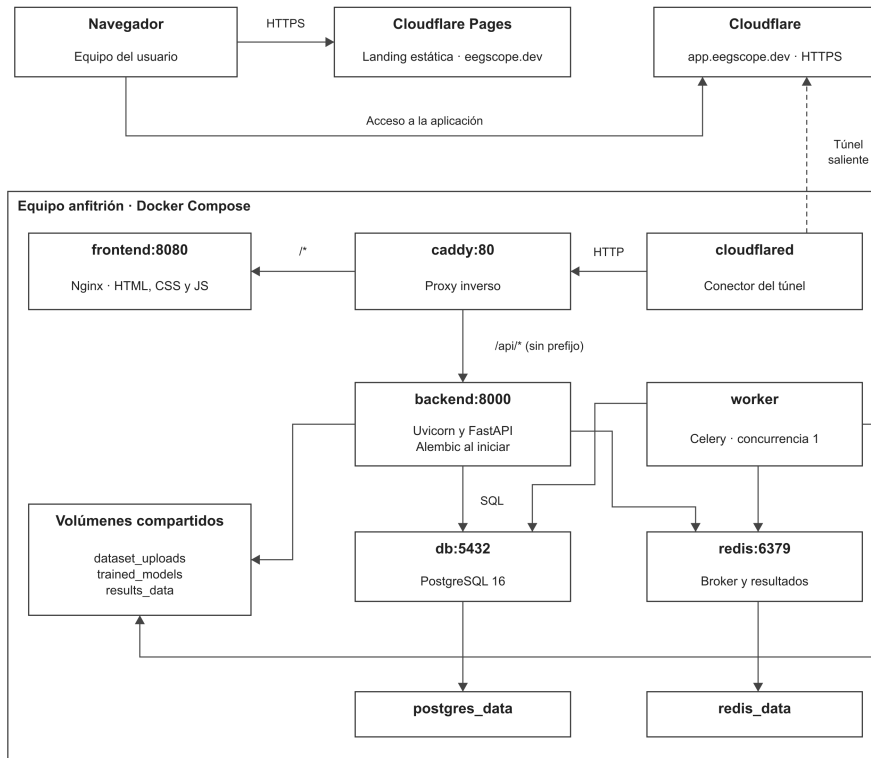


Figura C.5: Configuración de despliegue con Docker Compose, Cloudflare Tunnel y landing independiente en Cloudflare Pages.

El entorno local se levanta con `docker compose up -build`, sin instalar manualmente las dependencias de cada servicio. La publicación mediante el túnel requiere además la configuración de `compose.prod.yml`, las variables de entorno y el dominio. Los pasos se detallan en el manual del programador.

C.4. Diseño procedimental

En esta sección se documentan los procesos clave del sistema mediante diagramas de secuencia. Cada proceso refleja cómo interactúan los actores y los componentes para llevar a cabo una operación concreta.

Flujo de entrenamiento

Cuando el usuario lanza un entrenamiento desde la pestaña “Entrenamiento”, el sistema ejecuta la siguiente secuencia:

1. El usuario selecciona el dataset y los parámetros de entrenamiento.
2. El frontend envía al endpoint `POST /training/run` el CSV o el identificador del dataset guardado, junto con los parámetros seleccionados.
3. El backend comprueba el usuario y el acceso al dataset, registra el propietario del trabajo y lo envía a Celery. Devuelve `202 Accepted` con su identificador.
4. El worker llama a `training/service.py`. Los runners preparan los datos y ejecutan la validación cruzada *cross-subject* con `StratifiedGroupKFold`.
5. El sistema calcula las métricas globales, la matriz de confusión y el `classification_report`.
6. Se guardan el dataset, el experimento y las métricas por pliegue en la base de datos.
7. Se entrena y exporta el modelo final y se registra su relación con el experimento.
8. El frontend consulta `/tasks/{task_id}` hasta obtener el resultado o un error.
9. La interfaz muestra las métricas y si se han guardado el experimento y el modelo.

El diagrama representa el recorrido normal de un entrenamiento. La ejecución del worker y las consultas de estado ocurren en paralelo. Las operaciones sobre la base de datos y los ficheros se resumen dentro del servicio que las realiza; las rutas se muestran sin el prefijo público `/api`.

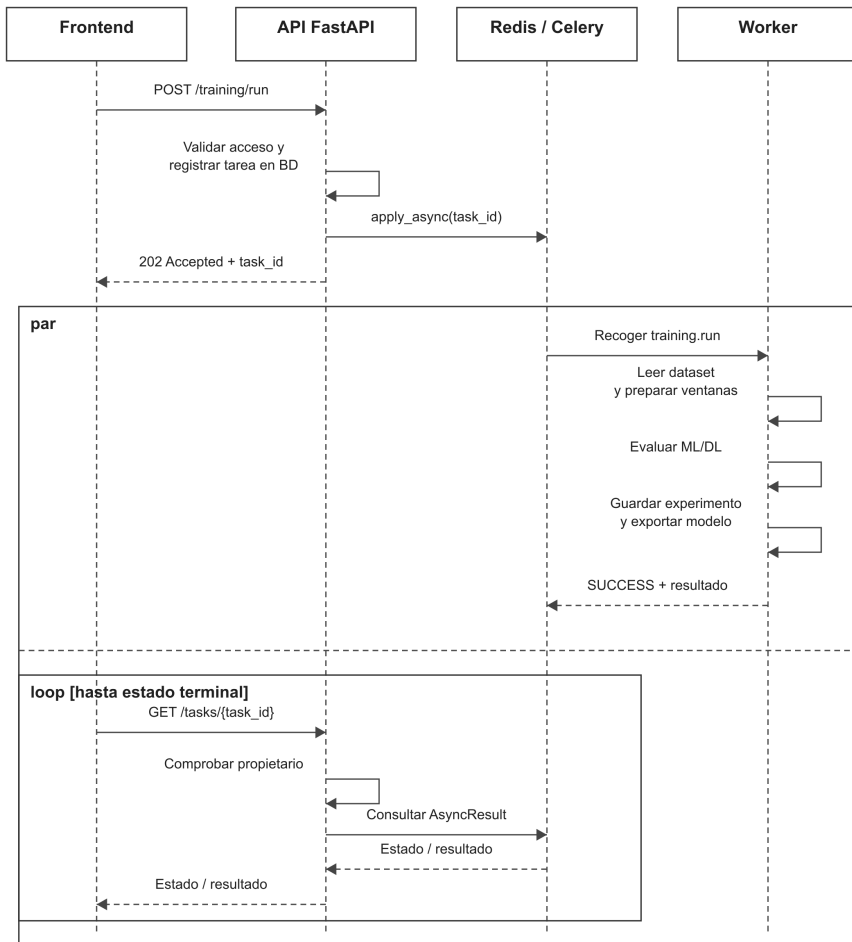


Figura C.6: Entrenamiento asíncrono con Celery y consulta periódica de su estado desde el frontend.

Durante el entrenamiento se puede navegar entre pestañas. La tarea se ejecuta en el worker y no depende de mantener abierta la vista de entrenamiento. La configuración actual procesa una tarea cada vez; las siguientes esperan en la cola.

Flujo de análisis del dataset

Cuando el usuario carga o selecciona un dataset de entrenamiento, el sistema realiza una comprobación previa antes de permitir el entrenamiento.

1. El usuario sube un CSV o selecciona un dataset previamente guardado.
2. Si es nuevo, el frontend lo sube y el backend lo guarda por hash SHA-256, registrando el acceso del usuario.
3. El frontend solicita el análisis mediante `POST /training/datasets/{id}/analysis`.
4. El backend comprueba el acceso y devuelve el identificador de la tarea encolada.
5. El worker calcula las estadísticas y el frontend consulta el estado hasta recibirlas.
6. La interfaz muestra el resumen del dataset y la información de pacientes.

Flujo de consulta de experimentos

El histórico de experimentos permite revisar los entrenamientos realizados desde la interfaz.

1. El usuario accede a la pestaña “Experimentos”.
2. El frontend solicita al backend el listado de experimentos del usuario autenticado.
3. El backend consulta la base de datos y devuelve los experimentos junto con la información resumida del dataset asociado.
4. La interfaz muestra la tabla y el mejor modelo disponible del usuario, ordenado por exactitud balanceada y, en caso de empate, por F1-score.

5. Al seleccionar un experimento, el frontend solicita su detalle.
6. El backend devuelve la configuración, las métricas y los resultados por pliegue.

Flujo de inferencia

El flujo de inferencia permite clasificar un CSV de un paciente usando uno de los modelos exportados.

1. El usuario selecciona uno de sus modelos entrenados o un modelo de referencia ML/DL.
2. El usuario sube un CSV de paciente.
3. El frontend envía el fichero al endpoint de validación.
4. El backend comprueba que el CSV contiene los canales esperados por el modelo seleccionado.
5. Si el fichero es válido, el usuario ejecuta la predicción.
6. El backend segmenta la señal en epochs y aplica el preprocesamiento correspondiente al modelo.
7. El modelo predice cada epoch y el sistema agrega los votos para obtener una clase final.
8. El frontend muestra la clasificación final, la confianza y la distribución de epochs por clase.

La confianza resume la salida del modelo sobre las ventanas. No representa una probabilidad clínica de que el paciente tenga TDAH. La inferencia se realiza durante la petición HTTP; los trabajos trasladados a Celery son el análisis de datasets y los entrenamientos de la aplicación.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

En este apéndice se aporta la información técnica necesaria para que un desarrollador pueda **comprender, instalar, modificar y desplegar** el proyecto. Se divide en cuatro partes: la estructura del repositorio, el manual del programador, los pasos de compilación, instalación y ejecución del proyecto y las pruebas del sistema.

D.2. Estructura de directorios

El repositorio tiene tres bloques: el **código de la aplicación** (backend, frontend, base de datos), los **recursos de investigación** (scripts, notebooks) y los **artefactos generados** (modelos, resultados, figuras). A continuación se muestra el árbol del directorio raíz con una breve descripción.

```
eeg-adhd-clasificacion/
├── backend/ ..... API FastAPI
├── frontend/ ..... SPA en React + Vite
├── landing/ ..... Página pública independiente
├── alembic/ ..... Migraciones de la base de datos
├── scripts/ ..... Parte de investigación y entrenamiento
├── notebooks/ ..... Hiperparámetros
├── tests/ ..... Tests pytest
└── models/ ..... Modelos exportados (.joblib y .keras)
```

results/	Resultados de CV (CSV y JSON)
Figuras/	PNGs generados
data/	CSV del dataset
docs/	Documentación
.github/	Workflows de CI (GitHub Actions)
alembic.ini	Configuración de Alembic
compose.yml	Servicios para ejecución local
compose.prod.yml	Configuración de despliegue
Caddyfile	Rutas del proxy inverso
pyproject.toml	Dependencias y herramientas de Python
uv.lock	Versiones resueltas de las dependencias
sonar-project.properties	Configuración de SonarCloud
.dockerignore	Ficheros excluidos del contexto Docker
.gitignore	Ficheros excluidos del control de versiones
.env.example	Plantilla de variables de entorno
LICENSE	Licencia MIT del proyecto
README.md	Información general y guía rápida de arranque

D.3. Manual del programador

En este apartado se describe la organización interna del sistema desde el punto de vista de un desarrollador. Se explican las principales capas de la aplicación, la responsabilidad de cada módulo y el flujo recomendado para añadir o modificar funcionalidades.

Arquitectura general de la aplicación

La aplicación sigue una arquitectura cliente-servidor. El cliente está implementado como una SPA desarrollada con React y Vite, mientras que el servidor expone una API REST mediante FastAPI. El almacenamiento de los datos se realiza en PostgreSQL y las migraciones se gestionan con Alembic.

El análisis de datasets y los entrenamientos lanzados desde la interfaz se ejecutan en un worker de Celery, usando Redis como cola de mensajes y almacén temporal de resultados [2]. De esta manera, la petición HTTP no tiene que esperar a que termine el entrenamiento y el usuario puede seguir navegando por la aplicación.

El frontend se encarga de la interacción con el usuario: carga de datasets en CSV o ZIP, visualización del dataset, selección de modelos, lanzamiento

de entrenamientos, consulta de experimentos y ejecución de predicciones. El backend concentra la lógica de validación, procesamiento, entrenamiento, inferencia y almacenamiento de resultados.

La comunicación entre ambas partes se realiza mediante peticiones HTTP con intercambio de datos en formato JSON. En el caso de la carga de ficheros EEG, el frontend utiliza peticiones `multipart/form-data`. Para entrenamiento puede enviar un CSV o un ZIP con un único CSV; para predicción se utiliza un CSV. El backend recupera los datos, valida su formato y los procesa antes de devolver una respuesta estructurada.

Organización del backend

El backend se encuentra en el directorio `backend/`. El punto de entrada es el fichero `backend/main.py`, donde se crea la aplicación FastAPI, se configura CORS, se sirven los ficheros estáticos de figuras y se registran los routers principales.

La organización del backend sigue una separación por dominios funcionales:

- `auth/`: registro, inicio de sesión, contraseñas y validación del usuario.
- `datasets/`: carga, almacenamiento y análisis de datasets.
- `training/`: configuración, evaluación y guardado de los modelos entrenados.
- `experiments/`: consulta y persistencia de experimentos.
- `model_registry/`: catálogo y registro de modelos disponibles.
- `inference/`: validación de CSV y predicción con modelos exportados.
- `worker/`: configuración de Celery y registro del propietario de cada tarea.
- `api/`, `core/`, `db/` y `modeling/`: respuestas comunes, configuración, modelos de datos y fábricas de ML/DL.

Dentro de cada dominio se separan los ficheros `router.py`, `schemas.py`, `service.py` y `repository.py` cuando son necesarios. Esta separación evita que los endpoints acumulen demasiada lógica. Los routers actúan como capa de entrada, validan la petición y delegan el trabajo en los servicios correspondientes.

Routers de la API

Los routers agrupan las operaciones de la API por funcionalidad. Los principales módulos son:

- `api/health.py`: permite comprobar que el backend está activo.
- `auth/router.py`: registro, login y consulta del usuario actual.
- `model_registry/router.py`: modelos disponibles, información y métricas.
- `inference/router.py`: validación del CSV y predicción final.
- `datasets/router.py`: carga de datasets y solicitud de análisis.
- `training/router.py`: opciones de entrenamiento y envío del trabajo a Celery.
- `experiments/router.py`: consulta de los experimentos del usuario.
- `api/tasks.py`: consulta del estado y resultado de una tarea propia.

FastAPI genera automáticamente la documentación interactiva de estos endpoints en `/docs`, lo que facilita probar la API durante el desarrollo.

EEG ADHD Classifier API GET GA115			
OpenAPI spec			
API para clasificación ADHD/Control a partir de señales EEG.			
Authorize			
default ^			
GET	/health	Health	
GET	/models	List Available Models	
GET	/models/best	Best Available Model	
GET	/model/info	Model Info	
GET	/model/figures	Model Figures	
POST	/validate	Validate File	
POST	/predict	Predict	
auth ^			
POST	/auth/register	Register	
POST	/auth/login	Login	
GET	/auth/me	Get Profile	
tasks ^			
GET	/tasks/{task_id}	Task Status	
training ^			
GET	/training/options	Training Options	
POST	/training/run	Training Run	
experiments ^			
GET	/experiments	Experiments List	
GET	/experiments/{experiment_id}	Experiment Detail	
datasets ^			
GET	/training/datasets	Training Datasets	
POST	/training/datasets	Upload Training Dataset	
POST	/training/datasets/{dataset_id}/analysis	Queue Dataset Analysis	
GET	/training/datasets/{dataset_id}/stats	Saved Training Dataset Stats	
POST	/training/dataset/stats	Training Dataset Stats	

Figura D.1: Documentación interactiva de la API, incluidos los endpoints de autenticación, trabajos asíncronos y datasets.

Servicios del backend

Los servicios se encuentran dentro de su dominio y contienen la lógica principal. Por ejemplo, `training/service.py` coordina la evaluación y el guardado del experimento, mientras que `training/runners.py` ejecuta los modelos ML y DL. Los ficheros `tasks.py` de datasets y entrenamiento permiten llamar a esta lógica desde Celery.

Separar la lógica en servicios tiene varias ventajas. En primer lugar, permite reutilizar funciones desde distintos endpoints. En segundo lugar, facilita la escritura de pruebas unitarias. Por último, mejora la mantenibilidad del código, ya que los routers quedan centrados únicamente en recibir peticiones y devolver respuestas.

En `datasets/service.py` se comprueba si el archivo de entrenamiento es un ZIP. En ese caso se exige un único fichero CSV, sin contraseña y con un tamaño descomprimido máximo de 512 MiB. Se comprueba el tamaño declarado y se limita la lectura del contenido. El fichero se lee en memoria sin extraer las rutas del ZIP al disco, y se conserva solo el nombre del CSV.

Después se aplica la misma validación que a un CSV enviado directamente. El almacenamiento y el hash se calculan sobre el contenido descomprimido, de modo que comprimir el mismo CSV no crea un dataset distinto. Los permisos de acceso se gestionan por usuario y no dependen de que el fichero se haya subido comprimido.

Modelos y procesamiento EEG

La lógica relacionada con Machine Learning y Deep Learning se distribuye entre `backend/modeling/` y `scripts/`. El directorio `scripts/` contiene el pipeline de investigación: carga de datos, preprocesado, segmentación en epochs, extracción de características, entrenamiento, evaluación y exportación de modelos.

Los modelos clásicos de Machine Learning trabajan sobre características extraídas de la señal EEG, mientras que los modelos de Deep Learning utilizan ventanas de señal como entrada. Los modelos exportados se almacenan en `models/`, junto con ficheros de metadatos y métricas que permiten consultar información desde la interfaz.

Tras evaluar un modelo desde la aplicación, se entrena una versión final y se guarda en `models/trained/`. PostgreSQL almacena la ruta y los metadatos, no el fichero binario. La inferencia se resuelve desde `inference/predictors.py`, tanto para estos modelos como para los dos modelos de

referencia incluidos en el proyecto. Guardar el experimento y guardar el modelo son operaciones distintas: pueden conservarse las métricas aunque falle la exportación final.

La función `make_group_kfold_splits`, en `scripts/split.py`, genera las particiones de validación cruzada de una en una mediante un iterador. Así se evita mantener una lista con copias de los datos de todas las particiones. Se conserva el uso de `StratifiedGroupKFold`, la semilla y la separación por sujeto. Los consumidores de esta función deben recorrer el iterador sin convertirlo en una lista si quieren mantener esta reducción del consumo de memoria.

Base de datos y migraciones

La aplicación utiliza PostgreSQL para almacenar usuarios, accesos a datasets, experimentos, modelos registrados y propietarios de tareas. La conexión y los modelos SQLAlchemy se encuentran en `backend/db/`; las consultas están en los repositorios de cada dominio.

El esquema de la base de datos se gestiona con Alembic. Las migraciones se almacenan en `alembic/versions/`. Al arrancar el backend mediante Docker Compose se ejecuta automáticamente:

```
alembic upgrade head
```

Esto garantiza que la base de datos tenga el esquema actualizado antes de iniciar la API. Si se añaden nuevas tablas o columnas, debe generarse una nueva migración y comprobarse su aplicación en un entorno limpio.

Organización del frontend

El frontend se encuentra en `frontend/` y está desarrollado con React, TypeScript y Vite. El punto de entrada es `src/main.tsx`. En `src/app/`, `AppRouter.tsx` organiza las rutas de acceso, registro, perfil y aplicación, y `App.tsx` agrupa las vistas principales.

La estructura principal del frontend es:

- `app/`: navegación, cabecera y composición de las vistas.
- `features/`: autenticación, datasets, entrenamiento, experimentos e inferencia. Cada funcionalidad reúne sus componentes, tipos, llamadas a la API y hooks.

- `shared/`: componentes, estilos, tipos y utilidades compartidas.
- `i18n/`: configuración de idiomas y traducciones.

Las llamadas de cada funcionalidad están en su fichero `api.ts` y utilizan el cliente común `shared/api/client.ts`. Este cliente añade el token de acceso y trata los errores de la API. El seguimiento de los trabajos se comparte desde `shared/api/tasks.ts`.

Autenticación y tareas en segundo plano

El registro guarda el correo y un hash de la contraseña mediante Argon2. El login devuelve un JWT firmado con HS256 y con fecha de caducidad. Los endpoints privados comprueban el token y el acceso al recurso solicitado. Los modelos entrenados pertenecen al usuario a través del experimento; los modelos de referencia son compartidos.

El frontend conserva el token en `sessionStorage` y consulta `/auth/me` para recuperar la sesión. Cerrar sesión elimina el token del navegador, pero no equivale a revocarlo en el servidor. No se ha implementado recuperación de contraseña ni renovación mediante refresh tokens.

Al encolar un trabajo, la API devuelve 202 `Accepted` con un `task_id`. El frontend consulta `/tasks/{task_id}` para mostrar los estados `PENDING`, `STARTED`, `SUCCESS` o `FAILURE`. PostgreSQL registra el propietario, mientras que Redis conserva temporalmente el resultado de Celery. En la configuración actual este resultado caduca tras una hora; los experimentos guardados permanecen en PostgreSQL.

El hook `useTrainingTask` conserva un resumen del modelo, el dataset y el número de pacientes enviados al iniciar un entrenamiento. Este resumen se guarda en `sessionStorage` junto con el identificador de la tarea para recuperar su seguimiento tras una recarga. La barra de estado utiliza estos datos en lugar de la selección actual del formulario. Al recibir un estado final se elimina la información de seguimiento del almacenamiento de sesión; el resultado recibido permanece en el estado de la interfaz mientras esta sigue abierta.

Flujo recomendado para añadir funcionalidad

Para añadir una nueva funcionalidad al sistema se recomienda seguir un flujo ordenado:

1. Definir qué datos necesita recibir y devolver la nueva operación.
2. Crear o modificar los esquemas Pydantic del dominio correspondiente.
3. Implementar la lógica en su servicio y las consultas en su repositorio.
4. Exponer la operación en su router, comprobando el usuario y sus permisos.
5. Si es una operación larga, definir su tarea de Celery y la consulta del resultado.
6. Añadir la función y sus tipos en el `api.ts` de la funcionalidad del frontend.
7. Crear o modificar los componentes React necesarios.
8. Añadir pruebas unitarias o de integración.
9. Comprobar el funcionamiento desde la interfaz y desde Swagger.

Este flujo mantiene la separación entre interfaz, API, lógica de negocio y persistencia.

Criterios de mantenimiento

Durante el mantenimiento del proyecto se deben respetar las responsabilidades de cada capa. Los componentes React no deben contener lógica compleja de procesamiento EEG, ya que esta debe permanecer en el backend o en los scripts especializados. Del mismo modo, los routers de FastAPI no deben implementar directamente algoritmos de entrenamiento o inferencia, sino delegar estas tareas en servicios.

También se recomienda mantener actualizadas las pruebas cuando se modifique una parte crítica del sistema, especialmente si afecta a la validación de datos, entrenamiento, inferencia o almacenamiento de experimentos.

D.4. **Compilación, instalación y ejecución del proyecto**

El sistema se distribuye con Docker Compose, lo que permite levantarlo entero con un único comando sin instalar por separado las dependencias de la aplicación en el sistema anfitrión, una vez disponible Docker.

Requisitos previos

- **Docker Desktop** o equivalente con Docker Engine y Docker Compose v2.
- **Memoria suficiente para el entrenamiento y el sistema anfitrión.** En las comprobaciones del despliegue se asignaron 8 GB al entorno de Docker sobre WSL2, en un equipo con 16 GB de RAM. Esta es la configuración utilizada, no un mínimo establecido mediante pruebas de carga. El consumo depende del dataset, la segmentación y el modelo.
- Navegador moderno (Chrome, Firefox, Edge) para acceder a la interfaz.

Instalación y arranque

Clonar el repositorio:

```
git clone https://github.com/ajg1005/eeg-adhd-clasificacion.  
git
```

Entrar en la carpeta del proyecto y crear el fichero de variables de entorno a partir de la plantilla:

```
cd eeg-adhd-clasificacion  
cp .env.example .env
```

Editar el fichero `.env` para establecer una contraseña segura en `POSTGRES_PASSWORD` y un secreto aleatorio de al menos 32 caracteres en `JWT_SECRET_KEY`. Las demás variables se toman de la plantilla. Este fichero no debe subirse al repositorio. Después, levantar los servicios con:

```
docker compose up --build
```

Docker Compose levanta cinco servicios:

- `db`: PostgreSQL 16 en el puerto 5432.
- `backend`: FastAPI + Uvicorn en el puerto 8000. Ejecuta `alembic upgrade head` antes de aceptar peticiones.

- **frontend**: aplicación React construida con Vite y servida mediante Nginx. Se expone en el puerto 5173 del sistema anfitrión y usa internamente el puerto 8080 del contenedor.
- **redis**: cola de mensajes y resultados temporales de Celery, sin puerto publicado al anfitrión.
- **worker**: proceso Celery que ejecuta el análisis y los entrenamientos. Está configurado con concurrencia uno.

Backend y worker comparten los volúmenes de datasets y modelos entrenados. Al modificar las dependencias o el código incluido en la imagen del worker hay que reconstruirla. El worker no recarga los cambios automáticamente.

En Windows con Docker Desktop sobre WSL2, la memoria disponible para los contenedores puede estar limitada aunque el equipo tenga más RAM. Si un entrenamiento termina por falta de memoria, hay que revisar los logs del worker y la memoria asignada a WSL2. Aumentar ese límite requiere dejar memoria disponible para el sistema anfitrión; no sustituye la revisión del consumo del entrenamiento.

Dependencias y ejecución fuera de Docker

Las dependencias de Python, pytest, Ruff y cobertura se configuran en `pyproject.toml`. El proyecto utiliza Python 3.13 y el fichero `uv.lock` fija las versiones resueltas [1]. Con uv instalado, el entorno local se prepara desde la raíz del repositorio:

```
uv sync --locked --no-build
```

Los scripts de investigación se ejecutan como módulos para que funcionen los imports del proyecto. Por ejemplo:

```
uv run --locked --no-build python -m scripts.train_ml
```

Para ejecutar los scripts de investigación es necesario descargar el dataset indicado en el README y guardar el fichero `adhdata.csv` en la carpeta `data/`, situada en la raíz del repositorio. Este fichero no se incluye en el código fuente.

No es necesario ejecutar estos scripts para utilizar los modelos de referencia ya exportados. La aplicación web permite, además, entrenar modelos propios a partir de los datasets cargados por el usuario.

Despliegue público

La configuración de `compose.prod.yml` añade Caddy y Cloudflare Tunnel. El túnel dirige el tráfico de <https://app.eegscope.dev> a Caddy; este envía `/api/` al backend y el resto al frontend. La configuración de producción no publica los puertos de PostgreSQL, frontend ni backend en el equipo anfitrión.

Para utilizarla hay que configurar el túnel de Cloudflare, dirigir su hostname al servicio `http://caddy:80` y añadir su token a `CLOUDFLARE_TUNNEL_TOKEN` en `.env`. Después se ejecuta:

```
docker compose -f compose.yml -f compose.prod.yml up -d --build
```

El túnel permite acceder al equipo que ejecuta los contenedores, pero no traslada el entrenamiento a Cloudflare [4]. Ese equipo debe permanecer encendido y conectado. La página pública de `landing/` se despliega por separado en Cloudflare Pages, con directorio raíz `landing`, comando `npm run build` y salida `dist` [3].

La carga pública está sujeta al límite de tamaño de las peticiones de Cloudflare. Este límite se aplica al fichero enviado y es distinto del máximo de 512 MiB establecido por la aplicación para el CSV dentro de un ZIP. Por este motivo, para el dataset completo se utiliza la carga comprimida. Si el proxy rechaza la petición con HTTP 413, el frontend muestra una indicación para comprimir el CSV o utilizar un fichero más pequeño. La compresión reduce el tamaño transferido, pero no elimina el límite ni reduce por sí sola la memoria necesaria al procesar el CSV.

Si el dominio público no está accesible, puede utilizarse la configuración local descrita anteriormente. Esta funciona sin Cloudflare Tunnel y requiere reconstruir el frontend con la URL local de la API. Para pasar de producción a local en el mismo equipo, primero se detiene la configuración de producción sin eliminar los volúmenes y después se arranca solo `compose.yml`:

```
docker compose -f compose.yml -f compose.prod.yml down
docker compose -f compose.yml up -d --build
```

En este caso, `VITE_API_BASE_URL` y `CORS_ORIGINS` deben conservar los valores locales de la plantilla. La interfaz estará disponible en <http://localhost:5173>. No debe utilizarse la opción `-v` al detener los servicios si se quieren conservar los volúmenes. Una instalación nueva en otro equipo no incluye las cuentas ni los datasets del despliegue público.

Verificación

En la configuración local, una vez levantados los contenedores, comprobar que todo responde:

- Frontend: <http://localhost:5173>.
- API y documentación interactiva (Swagger): <http://localhost:8000/docs>.
- Endpoint de *liveness*: <http://localhost:8000/health> debe devolver 200 OK.

Después, crear una cuenta, iniciar sesión, analizar un dataset y lanzar un entrenamiento. Comprobar que se puede cambiar de pestaña y que el experimento aparece al terminar. Si el modelo se ha guardado, debe poder seleccionarse para inferencia. También conviene comprobar la carga del mismo CSV directamente y dentro de un ZIP, su reutilización desde la lista y que la barra de entrenamiento conserva el modelo de la tarea al cambiar la selección del formulario o recargar la página. Para revisar el worker se puede utilizar `docker compose logs worker`. El endpoint de salud por sí solo no comprueba todo este flujo.

Apagado y limpieza

Para detener los contenedores sin perder datos:

```
docker compose down
```

Si se ha utilizado la configuración de producción, deben indicarse los mismos ficheros para detener también Caddy y el túnel:

```
docker compose -f compose.yml -f compose.prod.yml down
```

En la configuración local, para detenerlos y eliminar también los volúmenes (esto borra la base de datos, los datasets y los modelos guardados en esos volúmenes):

```
docker compose down -v
```

D.5. Pruebas del sistema

Las pruebas del sistema se han organizado en dos niveles complementarios: pruebas unitarias y pruebas de integración. Ambas se ejecutan con `pytest` y forman parte del pipeline de CI del proyecto.

Su objetivo principal es comprobar el comportamiento del backend, la autenticación, el aislamiento entre usuarios, las tareas asíncronas y el pipeline de procesamiento EEG.

Tipos de pruebas

El proyecto incluye pruebas unitarias y pruebas de integración. Las pruebas unitarias verifican funciones concretas, como la validación de ficheros CSV, el cálculo de métricas, la preparación de datos o la carga de modelos entre otras. Las pruebas de integración comprueban el comportamiento de distintos módulos trabajando conjuntamente, especialmente en los endpoints de la API y en los servicios del backend.

Las pruebas de carga comprimida comprueban que se conserva el contenido del CSV, se rechazan archivos ZIP inválidos, con varios ficheros o con un CSV que supera el tamaño permitido, y se utiliza solo el nombre del fichero interno. La prueba de integración comprueba que subir el mismo CSV comprimido reutiliza el dataset y mantiene el control de acceso entre usuarios. En las particiones de entrenamiento se comprueba que no se solapan sujetos entre entrenamiento y prueba y que el iterador no retiene los datos de una partición ya consumida al avanzar a la siguiente.

Ejecución local de las pruebas

Las pruebas pueden ejecutarse localmente con el siguiente comando:

```
uv run --locked --no-build pytest -q
```

Para obtener además el informe de cobertura se utiliza:

```
uv run --locked --no-build pytest -q -m "not slow" --cov
```

El informe mide el código de backend y scripts incluido en la configuración de cobertura; no representa toda la aplicación. El comando anterior excluye los tests marcados como `slow`. La cifra de cobertura y el número de tests deben corresponder a la ejecución mostrada, ya que cambian al modificar el código.

La ejecución de CI mostrada corresponde al commit `b118ff0`: registra 144 pruebas superadas y un 86 % de cobertura en el código medido. Los 12 ficheros vacíos omitidos del informe no son pruebas saltadas.

```

177 backend/auth/schemas.py 14 0 100%
178 backend/auth/security.py 31 5 84% 34, 61-62, 64, 78
179 backend/auth/service.py 31 3 90% 27-28, 46
180 backend/core/config.py 27 8 70% 18-26
181 backend/datasets/repository.py 82 14 81% 55-68, 77, 81, 122-129, 155-159, 199
182 backend/datasets/router.py 40 6 85% 51-52, 74-75, 103-104
183 backend/datasets/schemas.py 36 0 100%
184 backend/datasets/service.py 43 5 88% 18, 73-74, 83, 91
185 backend/datasets/tasks.py 6 0 100%
186 backend/db/_init_.py 2 0 100%
187 backend/db/engine.py 12 1 92% 11
188 backend/db/models.py 107 0 100%
189 backend/experiments/repository.py 33 1 97% 65
190 backend/experiments/router.py 17 0 100%
191 backend/experiments/schemas.py 52 0 100%
192 backend/experiments/service.py 18 0 100%
193 backend/inference/predictors.py 177 27 85% 90, 105, 131-132, 142, 147-149, 212, 215, 374-383, 387, 428-421, 502-504, 512, 523-526
194 backend/inference/router.py 28 6 79% 37-38, 63-66
195 backend/inference/schemas.py 18 0 100%
196 backend/inference/service.py 6 0 100%
197 backend/inference/upload.py 10 0 100%
198 backend/main.py 24 0 100%
199 backend/model_registry/catalog.py 12 0 100%
200 backend/model_registry/repository.py 26 0 100%
201 backend/model_registry/router.py 28 6 79% 48-49, 60-68
202 backend/model_registry/schemas.py 43 0 100%
203 backend/model_registry/service.py 24 1 96% 83
204 backend/modeling/common.py 46 21 54% 25, 28, 42, 55-99
205 backend/modeling/dl_factory.py 21 15 29% 24-26, 30-32, 43-61, 65-66
206 backend/modeling/model_factory.py 8 0 100%
207 backend/training/data.py 36 5 86% 29-31, 41, 71
208 backend/training/persistence.py 83 28 66% 62-71, 97, 147-225
209 backend/training/router.py 41 8 80% 24, 27-28, 30, 61, 72, 94-95
210 backend/training/runners.py 155 58 63% 80, 124-203, 307-308, 345-348, 366-377, 381-383
211 backend/training/schemas.py 53 0 100%
212 backend/training/service.py 62 10 84% 171-174, 192-195, 201, 215
213 backend/training/tasks.py 8 0 100%
214 backend/worker/celery_app.py 4 0 100%
215 backend/worker/job_repository.py 15 0 100%
216 backend/worker/job_service.py 15 0 100%
217 backend/worker/tasks.py 4 0 100%
218 scripts/constants.py 28 0 100%
219 scripts/epochs.py 17 0 100%
220 scripts/evaluation.py 17 0 100%
221 scripts/feature_pipeline.py 35 2 94% 23, 41
222 scripts/features.py 39 2 95% 17, 22
223 scripts/ml_model_registry.py 16 1 94% 93
224 scripts/pipeline.py 24 1 96% 96
225 scripts/preprocessing.py 18 4 78% 25, 33-34, 40
226 scripts/signal_preprocessing.py 26 21 19% 8-11, 25-40, 51-63
227 scripts/spectral_features.py 67 4 94% 24, 30, 60, 69
228 scripts/split.py 57 13 77% 51, 71-72, 75-76, 82-83, 115-116, 120-121, 128-129
229 scripts/validators.py 25 2 92% 42, 46
230 -----
231 TOTAL 1979 282 86%
232
233 12 empty files skipped.
234 Coverage XML written to file coverage.xml
235 ----- 144 passed in 11.65s -----

```

Figura D.2: Resultado de las pruebas en CI: 144 pruebas superadas e informe de cobertura de backend y scripts.

Integración continua y análisis estático

El repositorio incluye un flujo de integración continua mediante GitHub Actions. El trabajo de Python instala las dependencias con uv, ejecuta Ruff, lanza los tests con cobertura y envía el análisis a SonarCloud. El trabajo del frontend comprueba los tipos de TypeScript, ejecuta ESLint y construye la aplicación con Vite.

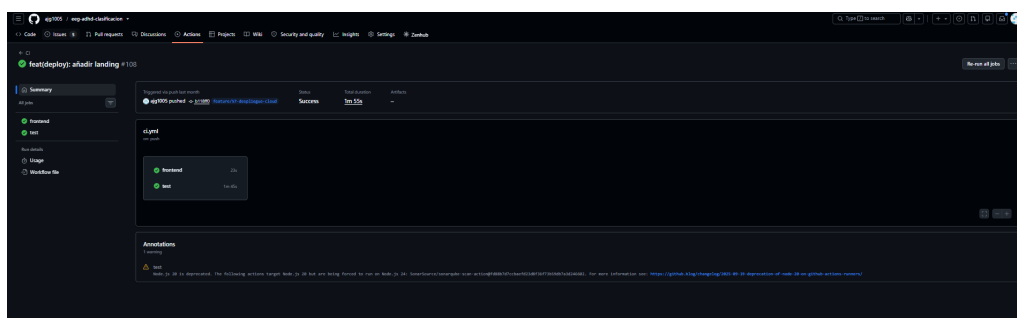


Figura D.3: Ejecución correcta de los trabajos frontend y test en GitHub Actions para el commit b118ff0.

SonarCloud se utiliza para revisar métricas de calidad del código, duplicación, cobertura y mantenibilidad. El frontend se analiza como código fuente, pero queda excluido del cálculo de cobertura por falta de tests de interfaz.

El panel de SonarQube Cloud corresponde al commit 4d538009 de main, posterior al merge, no a la misma revisión de las capturas de CI. Muestra el Quality Gate superado, un 86,56 % de cobertura y un 0 % de duplicación sobre código nuevo. No son métricas globales de toda la aplicación. También aparecen 47 incidencias nuevas; superar el Quality Gate no implica que no existan incidencias pendientes. Son evidencias de esas revisiones, no de una nueva ejecución sobre todos los cambios locales.

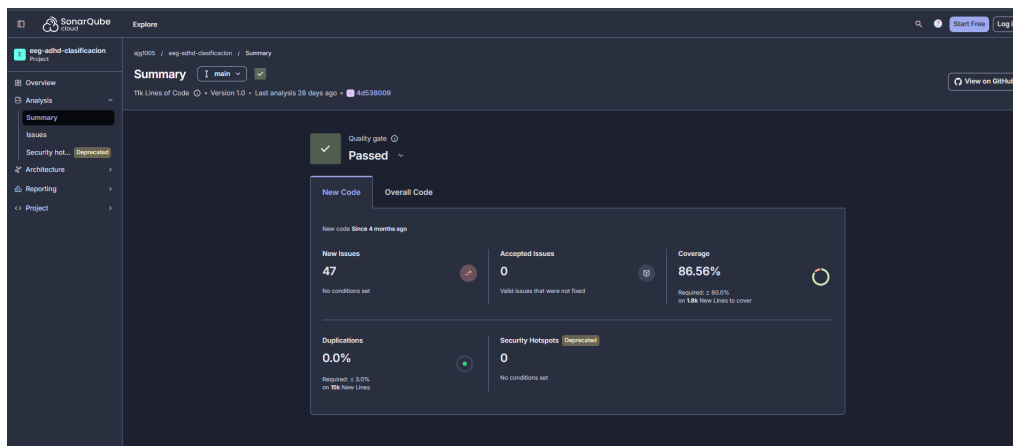


Figura D.4: Quality Gate superado y métricas de código nuevo en SonarQube Cloud.

Apéndice *E*

Documentación de usuario

E.1. Introducción

En este apéndice se describen los requisitos para utilizar la aplicación, junto con indicaciones de cómo acceder y utilizar cada una de sus funcionalidades. El manual está dirigido al usuario final, en este caso un investigador, que no tiene por qué tener conocimientos de programación.

E.2. Requisitos de usuarios

Los requisitos necesarios para utilizar la aplicación web son:

- Un navegador y una cuenta de usuario en la aplicación.
- Acceso a la aplicación ya sea levantada en local con Docker (<http://localhost:5173>) o en el servidor desplegado.
- Para entrenamiento, un CSV multipaciente con los 19 canales EEG, columna **ID** y columna **Class** con la etiqueta ADHD/Control. Este CSV también puede subirse dentro de un ZIP con un único fichero CSV y sin contraseña. Para predicción, un CSV de un único paciente con los canales esperados por el modelo; no hace falta conocer su clase. La frecuencia de muestreo de referencia es 128 Hz.
- Si se ejecuta en local, hay que disponer de memoria suficiente para Docker y para el resto del sistema. En las comprobaciones del despliegue se asignaron 8 GB al entorno de Docker; esta configuración no establece un mínimo de RAM para cualquier equipo. La memoria necesaria

depende del dataset y del modelo, sobre todo en Deep Learning. Si se accede por Internet, el cálculo se realiza en el equipo que aloja los contenedores.

E.3. Instalación

En cuanto a la instalación, la aplicación se utiliza directamente en el navegador. No es necesario instalarla en el equipo del usuario cuando se accede a una instancia ya desplegada.

Acceso desde un servidor

La dirección prevista para acceder a la aplicación es <https://app.eegscope.dev>. La página pública del proyecto está separada de la aplicación. En el despliegue mediante túnel, el equipo que aloja los contenedores debe estar encendido y conectado para poder utilizarla.

Para la evaluación del proyecto se ha preparado una cuenta de usuario con un dataset cargado. Las credenciales se facilitan al tribunal en el fichero `README.txt` de la raíz del USB de entrega. Esta cuenta permite acceder a las funcionalidades descritas en el manual sin tener que realizar el registro ni cargar inicialmente el dataset.

Levantar en local con Docker

En caso de no estar disponible el acceso al servidor también se podría levantar una instancia local:

1. Clonar el repositorio del proyecto.
2. Copiar el fichero `.env.example` a `.env`. Configurar `POSTGRES_PASSWORD` para la base de datos y un secreto aleatorio de al menos 32 caracteres en `JWT_SECRET_KEY` para las sesiones.
3. Levantar los servicios con `docker compose up --build`.
4. Abrir el navegador en <http://localhost:5173>.

La instancia local utiliza sus propios datos. En una instalación nueva hay que registrar una cuenta y cargar el dataset; la cuenta preparada para el tribunal pertenece al despliegue público y no se copia automáticamente.

Para ejecutar la aplicación en local no es necesario configurar Cloudflare Tunnel.

E.4. Manual del usuario

En esta sección se describe cómo utilizar las diferentes funcionalidades de la aplicación web. Primero se explica el acceso y después las acciones que se pueden realizar en cada pestaña.

Las capturas muestran la interfaz en inglés. Los nombres de las acciones se describen en español; el idioma se puede cambiar desde la cabecera.

Registro, inicio de sesión y perfil

Al abrir la aplicación aparece la pantalla de inicio de sesión. Si todavía no se tiene una cuenta, se puede acceder al registro e introducir el correo y la contraseña. No se permite registrar dos cuentas con el mismo correo.

Una vez iniciada la sesión, se accede a las pestañas de trabajo. Desde el perfil se puede consultar el correo, el estado de la cuenta y la fecha de registro, además de cerrar sesión. Si la sesión caduca, hay que volver a identificarse. Los experimentos y modelos propios no se comparten con otros usuarios.

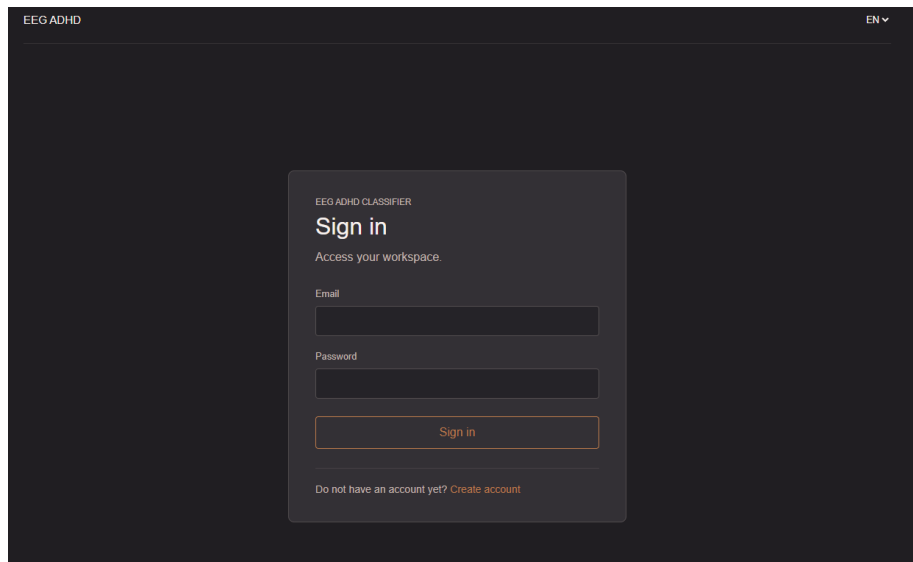


Figura E.1: Pantalla de inicio de sesión.

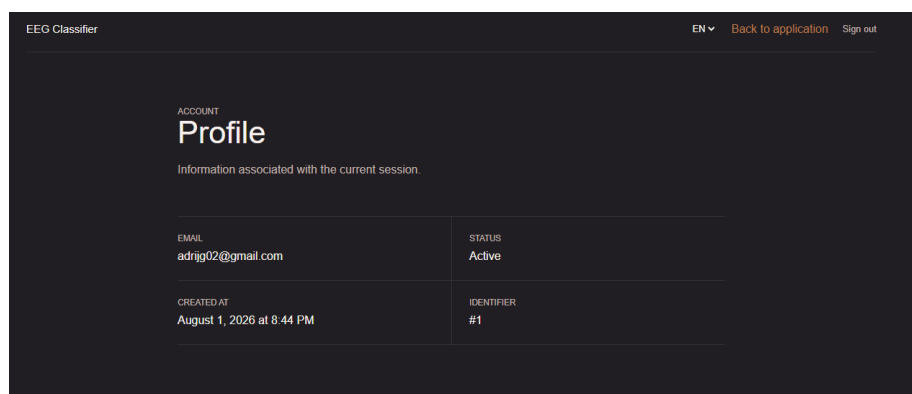


Figura E.2: Perfil de la cuenta utilizada en la demostración.

Vista general

La cabecera muestra la identificación de la aplicación y los accesos principales. Las vistas de Dataset, Entrenamiento y Experimentos forman el flujo de entrenamiento, mientras que Modelo de inferencia y Predicción permiten utilizar los modelos ya disponibles.

En la cabecera se encuentra el selector de idioma, que permite alternar entre español e inglés, y el acceso al perfil.

Pestaña Dataset de entrenamiento

La primera pestaña disponible es la que permite cargar el dataset para realizar el entrenamiento.

Se puede cargar un nuevo dataset pulsando en el cuadrado con el texto *Seleccionar CSV EEG o ZIP*. En caso de que ya haya datasets disponibles para la cuenta, también está la opción de elegir uno de la lista.

La carga admite un CSV directamente o un ZIP con un único fichero CSV, sin contraseña. El CSV contenido en el ZIP puede ocupar hasta 512 MiB descomprimido y debe mantener las mismas columnas y el mismo formato que en una carga sin comprimir. La compresión solo cambia la forma de enviar el fichero.

En el despliegue público, el archivo enviado debe respetar también el límite de subida de Cloudflare. Para cargar el dataset completo utilizado en el proyecto se recomienda comprimir `adhddata.csv` en ZIP: en la comprobación realizada pasó de unos 267 MB a 46 MB. Si la subida devuelve un error

HTTP 413, la interfaz indica que se ha superado el límite y recomienda comprimir el CSV o utilizar un archivo más pequeño. El ZIP también debe quedar por debajo del límite de subida.

El servidor recupera y guarda el CSV, por lo que puede reutilizarse después desde la lista sin volver a subir el ZIP. Si el contenido coincide con un dataset ya almacenado, se reutiliza el fichero y se registra el acceso correspondiente al usuario.

Al seleccionar un fichero se inicia su carga y, una vez guardado, se solicita el análisis en segundo plano. Al elegir un dataset de la lista también se inicia su análisis. Mientras se calcula, la interfaz muestra que se está analizando; si falla, aparece un mensaje y se puede utilizar el botón *Reintentar análisis*.

Al terminar se obtiene un resumen con las características del fichero (filas, columnas, sujetos, distribución de clases y canales detectados) y una tabla de pacientes que se puede filtrar por clase.

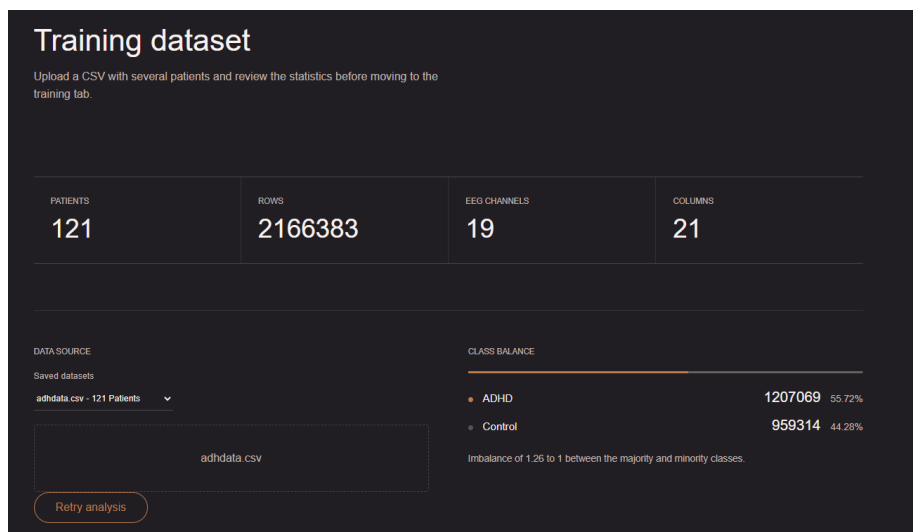


Figura E.3: Dataset analizado: 121 sujetos, 19 canales EEG y distribución de clases calculada por filas del CSV.

Pestaña Entrenamiento

En esta pestaña el usuario puede seleccionar los diferentes parámetros del entrenamiento.

Se pueden configurar tanto parámetros más relacionados con EEG (ventana, solape, frecuencia, bandas, etc.) como los hiperparámetros del modelo

y los parámetros generales del entrenamiento. Estos últimos varían según se elija un modelo de Machine Learning o de Deep Learning.

Una vez seleccionados los parámetros, se puede lanzar el entrenamiento *cross-subject* pulsando el botón de *Entrenar*. El trabajo se ejecuta en segundo plano y se puede cambiar de pestaña mientras termina. Si ya hay otra tarea ejecutándose, el entrenamiento queda pendiente en la cola.

La barra de estado muestra el modelo, el dataset y el número de pacientes asociados a la tarea enviada. Cambiar la selección del formulario no modifica el entrenamiento que ya está en curso. La información de la tarea se conserva al cambiar de pestaña y al recargar la página en la misma sesión del navegador.

The screenshot shows the 'Training' tab of the EEG Classifier application. The interface is dark-themed. At the top, a navigation bar includes links for 'Training dataset', 'Training' (active), 'Experiments', 'Inference model', and 'Prediction'. The user is logged in as 'adrijo2@gmail.com'. The main heading is 'Training' with a subtext: 'Configure the signal and the model, launch training and review the per-patient results.' Below this, there are two tabs: 'Machine Learning' (selected) and 'Deep Learning'. Under 'Machine Learning', several parameters are configured: 'Model to train' is 'XGBoost', 'Number of estimators' is '200', 'Maximum depth' is '4', 'Learning rate' is '0.05', 'Row subsampling' is '0.8', and 'Column subsampling' is '0.8'. Below these, 'Training parameters' are shown: 'Window size' is '1920', 'Window step' is '960', 'Sampling frequency' is '128', 'Samples per segment' is '960', 'Feature type' is 'Temporal and spectral', and 'Apply filtering' is 'No'. At the bottom, a status bar shows 'Dataset: adhdata.csv', 'Patients: 121', and 'Model: XGBoost'. A 'Train model' button is located in the bottom right corner.

Figura E.4: Configuración de un entrenamiento con XGBoost y características temporales y espectrales.

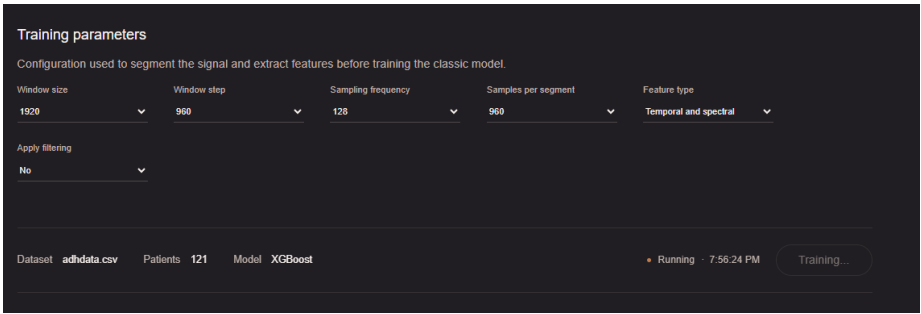


Figura E.5: Estado de ejecución del entrenamiento en segundo plano.

Cuando finaliza, se muestran las métricas agregadas, la matriz de confusión, una tabla con los resultados por paciente y, en el caso del Machine Learning, la importancia de las características.

La aplicación intenta guardar el experimento y exportar un modelo final para inferencia. Si falla el guardado del modelo, puede aparecer un aviso aunque las métricas del experimento se hayan guardado correctamente. En ese caso, ese entrenamiento no tendrá un modelo disponible para predecir.

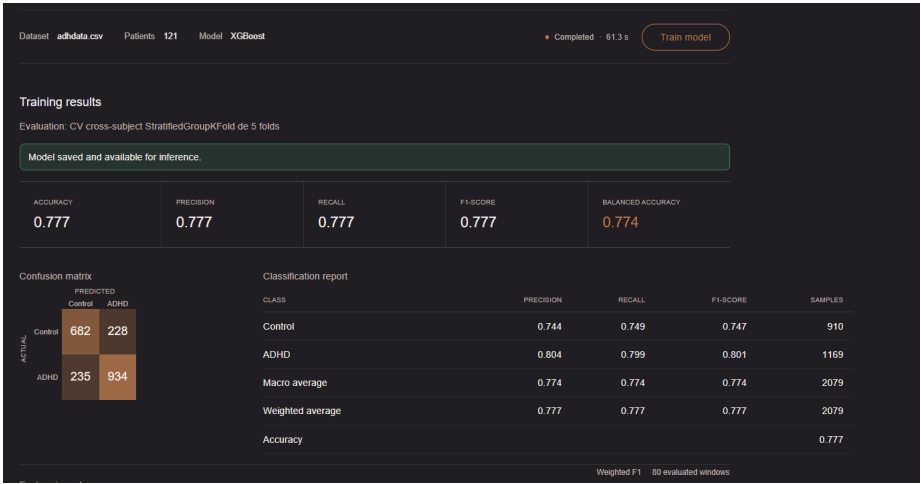


Figura E.6: Resultados de XGBoost: métricas, matriz de confusión y confirmación del guardado del modelo.

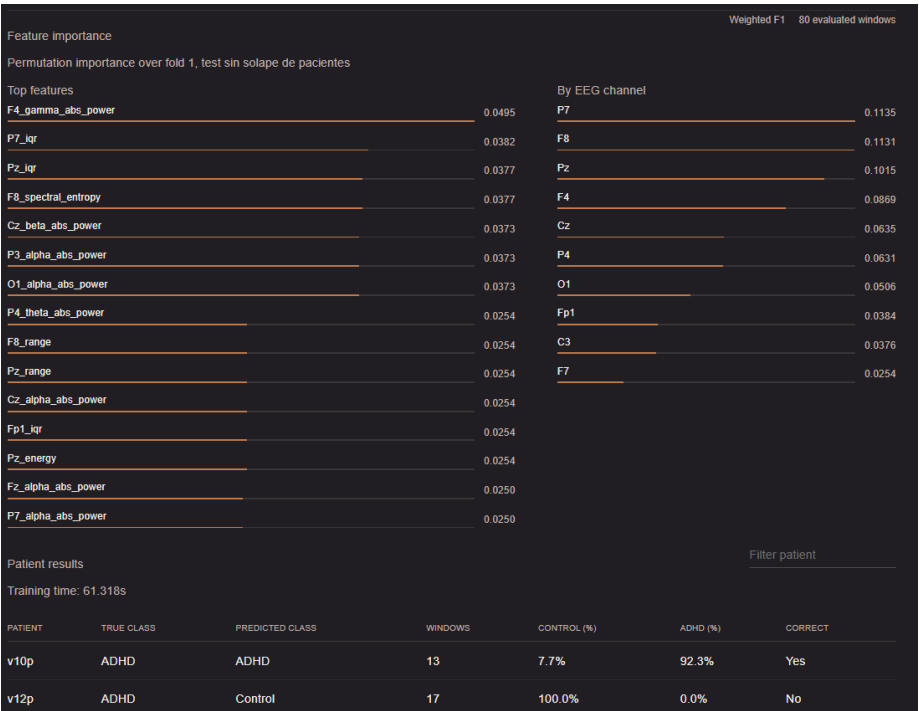


Figura E.7: Importancia por permutación de características y canales sobre 80 ventanas del conjunto de prueba del primer pliegue.

Pestaña Experimentos

En esta pestaña se puede consultar el histórico de entrenamientos del usuario. También se muestra el mejor modelo propio disponible, según la exactitud balanceada y, en caso de empate, el F1-score. Esta comparación no significa que un modelo sea mejor en cualquier situación, sobre todo si se han utilizado datasets distintos.

Al seleccionar un experimento en la tabla, se muestra el detalle con los parámetros utilizados y las métricas agregadas. Si tiene un modelo exportado, se puede elegir para utilizarlo en inferencia.

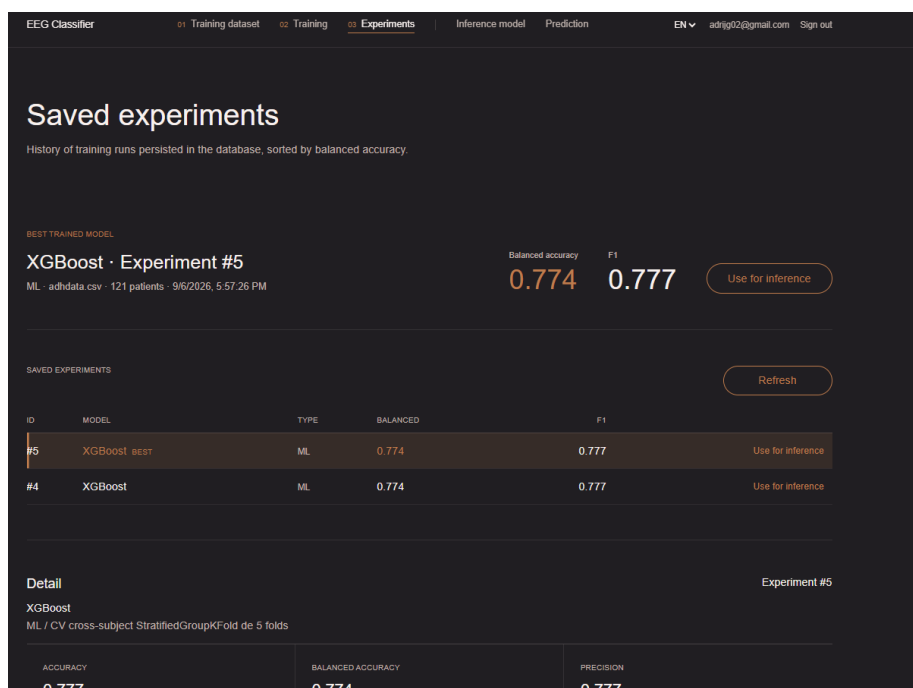


Figura E.8: Historial de experimentos y selección del mejor modelo propio disponible.

Pestaña Modelo de inferencia

En esta pestaña se puede consultar la información de los modelos propios y de los modelos de referencia de Machine Learning y Deep Learning. El usuario puede cambiar entre los disponibles mediante el desplegable.

Se muestra la información del modelo activo, sus parámetros y las métricas de validación. Cuando el modelo tiene figuras asociadas, también se pueden consultar. No todos los modelos entrenados desde la aplicación tienen las mismas figuras que los modelos de referencia.

El modelo seleccionado en esta pestaña es el que se utilizará posteriormente en la pestaña *Predicción* para clasificar nuevos pacientes.

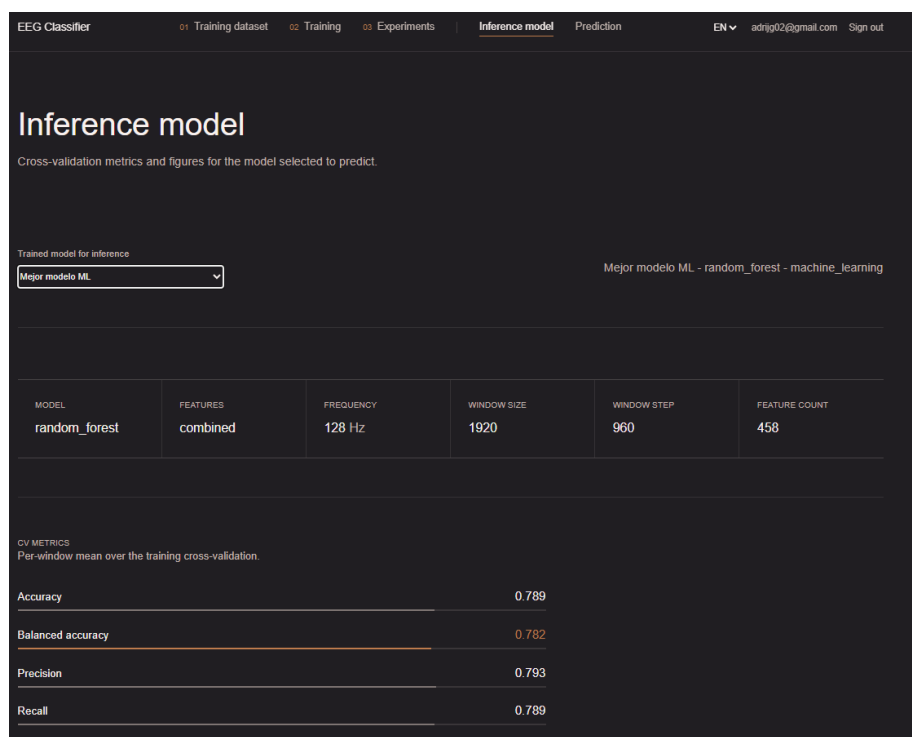


Figura E.9: Información y métricas del modelo de referencia de Machine Learning, Random Forest.

Pestaña Predicción

En esta pestaña, en el panel de *Archivo del paciente*, se puede subir un CSV con los datos de un paciente individual. Esta pestaña no admite ZIP. La aplicación valida el formato automáticamente y muestra la información del fichero (nombre, tamaño, filas y canales detectados) junto con los canales esperados por el modelo.

En la parte superior se puede seleccionar el modelo que se utilizará en la inferencia, eligiendo entre los modelos propios y los de referencia disponibles.

Al pulsar el botón *Ejecutar predicción*, en la parte derecha de la pantalla aparece el resultado: la clasificación final (TDAH o Control), la confianza media, el número de ventanas asignadas a cada clase y la distribución de las ventanas en formato numérico y en un pequeño gráfico.

La confianza es un resumen de las predicciones de las ventanas, no una probabilidad clínica de tener TDAH. El resultado debe interpretarse como una clasificación experimental, no como un diagnóstico.

En el ejemplo de predicción se utiliza el XGBoost del experimento 5, distinto del Random Forest de referencia mostrado en la figura anterior. Estas capturas ilustran el funcionamiento de la aplicación, no una evaluación independiente de su capacidad de generalización.

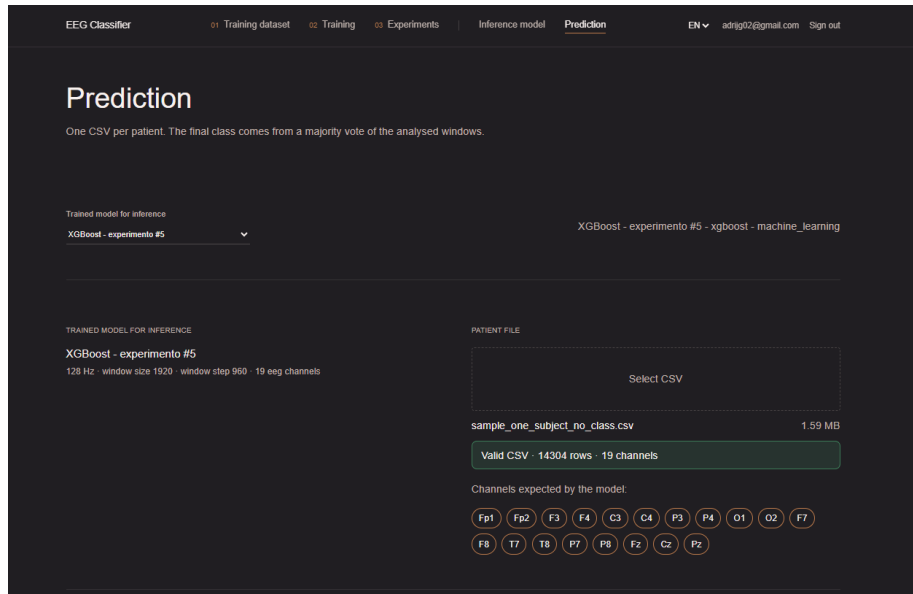


Figura E.10: Validación del CSV de un sujeto con el XGBoost del experimento 5 seleccionado.

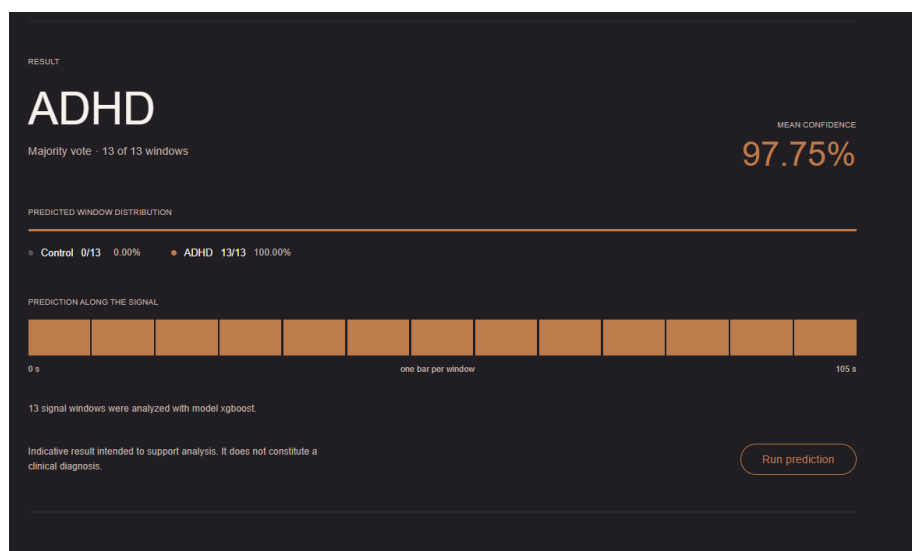


Figura E.11: Resultado de inferencia con XGBoost: clase asignada, confianza media y votos de las 13 ventanas analizadas.

Otras funcionalidades

Por último, el idioma se puede cambiar desde la cabecera. Si una operación falla, la interfaz muestra un mensaje con el problema. En el caso de un trabajo en segundo plano, conviene consultar primero su estado antes de volver a lanzarlo.

Anexo de sostenibilización curricular

F.1. Introducción

Este Trabajo de Fin de Grado ha consistido en el desarrollo de una aplicación web para la clasificación de señales EEG asociadas al TDAH mediante técnicas de Machine Learning y Deep Learning. Aunque el objetivo principal del proyecto es técnico, durante su desarrollo también se han tratado aspectos relacionados con la sostenibilidad, especialmente desde el punto de vista social, ético, económico y tecnológico.

En este proyecto, la sostenibilidad también afecta al mantenimiento del código, a la reutilización del trabajo y al cuidado con el que se presentan los resultados sobre datos de salud.

F.2. Sostenibilidad social y ética

Uno de los aspectos más importantes del proyecto es su relación con el ámbito de la salud. El sistema trabaja con señales EEG y con una clasificación binaria entre sujetos con TDAH y sujetos de control. Por este motivo, una decisión importante ha sido dejar claro que la aplicación no debe utilizarse como herramienta de diagnóstico clínico. El resultado generado por el modelo debe interpretarse como una salida experimental dentro de un prototipo académico, no como una decisión médica.

Explicar este límite ayuda a evitar falsas expectativas. Los resultados dependen de los datos, de cómo se han preparado las señales y de cómo se

ha evaluado el modelo; una buena métrica en este estudio no garantiza el mismo resultado con otros pacientes.

También se ha tenido en cuenta la protección de los datos. Los CSV de EEG no deben incluir nombres, DNI ni historia clínica. La aplicación incorpora cuentas con correo y contraseña, y comprueba el acceso a datasets, experimentos, modelos y tareas. Estas medidas no sustituyen una política de conservación de datos ni una revisión de las condiciones de uso de los datos reales.

F.3. Sostenibilidad tecnológica

Desde el punto de vista técnico, se ha intentado desarrollar una aplicación mantenible y reproducible. Para ello, el proyecto se ha organizado separando responsabilidades: backend, frontend, scripts de investigación, modelos exportados, base de datos y pruebas. Esta separación facilita que otra persona pueda entender el código, modificarlo o continuar el trabajo en el futuro.

También se ha utilizado Docker Compose para facilitar la ejecución del sistema. Esta decisión reduce problemas de instalación y mejora la portabilidad, ya que permite levantar la base de datos, el backend, el frontend, Redis y el worker con una configuración común. Esto contribuye a la sostenibilidad tecnológica porque evita depender de configuraciones manuales difíciles de reproducir.

La validación *cross-subject* mantiene separados los sujetos de entrenamiento y prueba. Así se evita que las ventanas de una misma persona aparezcan en ambos grupos y den una impresión demasiado favorable del rendimiento. Documentar esta decisión permite entender cómo se han obtenido las métricas.

F.4. Sostenibilidad económica y reutilización

El proyecto utiliza principalmente tecnologías libres o de uso abierto, como Python, FastAPI, React, PostgreSQL, scikit-learn y TensorFlow. Esto facilita la reutilización del código y reduce el coste de las herramientas utilizadas. La distribución y reutilización del proyecto deben respetar las licencias de sus componentes y del dataset.

Además, la aplicación permite guardar experimentos y reutilizar datasets de entrenamiento. Esta funcionalidad evita repetir trabajo de forma innecesaria y facilita comparar configuraciones, modelos y resultados. En un contexto académico, esto ayuda a aprovechar mejor el tiempo y los recursos disponibles.

F.5. Sostenibilidad ambiental

No se ha medido el consumo energético del proyecto, por lo que no se puede cuantificar su impacto ambiental. Sí se han considerado aspectos relacionados con el uso de recursos computacionales. El entrenamiento de modelos de Deep Learning puede exigir más tiempo y recursos que los modelos clásicos de Machine Learning. Por este motivo, se han comparado ambos enfoques y no se ha asumido que el modelo más complejo sea siempre el más adecuado.

La comparación entre modelos permite valorar el equilibrio entre rendimiento, coste e interpretación de los resultados. Además, ejecutar un entrenamiento mediante Celery permite seguir utilizando la aplicación, pero no reduce por sí solo el trabajo de cálculo. En el despliegue mediante túnel, ese cálculo sigue dependiendo del equipo que aloja los contenedores.

F.6. Reflexión final

El trabajo me ha llevado a tener en cuenta cómo se podría utilizar la aplicación y qué límites deben quedar claros. En este caso, eso supone explicar cómo se evalúan los modelos, controlar el acceso a los datos y facilitar que otra persona pueda continuar el proyecto. También queda pendiente medir el consumo de los entrenamientos para valorar su impacto ambiental con datos.

Bibliografía

- [1] Astral. uv: Working on projects. <https://docs.astral.sh/uv/guides/projects/>. Accedido: 05-09-2026.
- [2] Celery Project. Introduction to celery. <https://docs.celeryq.dev/en/stable/getting-started/introduction.html>. Accedido: 05-09-2026.
- [3] Cloudflare. Cloudflare pages: Git integration. <https://developers.cloudflare.com/pages/configuration/git-integration/>. Accedido: 05-09-2026.
- [4] Cloudflare. Cloudflare tunnel. <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>. Accedido: 05-09-2026.
- [5] Docker. Docker compose documentation. <https://docs.docker.com/compose/>, 2026. Accedido: 23-06-2026.
- [6] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Accedido: 23-06-2026.
- [7] GitHub. Github actions documentation. <https://docs.github.com/actions>, 2026. Accedido: 23-06-2026.
- [8] GitHub. Github docs. <https://docs.github.com/>, 2026. Accedido: 23-06-2026.
- [9] Jefatura del Estado. Ley orgánica 3/2018, de protección de datos personales y garantía de los derechos digitales. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>, 2018. Accedido: 23-06-2026.

- [10] Meta Open Source. React documentation. <https://react.dev/>, 2026. Accedido: 23-06-2026.
- [11] Open Source Initiative. The mit license. <https://opensource.org/licenses/mit/>, 2026. Accedido: 23-06-2026.
- [12] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. Accedido: 23-06-2026.
- [13] pytest. pytest documentation. <https://docs.pytest.org/>, 2026. Accedido: 23-06-2026.
- [14] pytest-cov. pytest-cov documentation. <https://pytest-cov.readthedocs.io/>, 2026. Accedido: 23-06-2026.
- [15] Redis. Licenses. <https://redis.io/legal/licenses/>. Accedido: 05-09-2026.
- [16] Software Freedom Conservancy. Git documentation. <https://git-scm.com/doc>, 2026. Accedido: 23-06-2026.
- [17] SonarSource. Sonarqube cloud documentation. <https://docs.sonarsource.com/sonarqube-cloud/>, 2026. Accedido: 23-06-2026.
- [18] Unión Europea. Reglamento (ue) 2016/679 del parlamento europeo y del consejo, reglamento general de protección de datos. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016. Accedido: 23-06-2026.
- [19] Unión Europea. Reglamento (ue) 2017/745 sobre los productos sanitarios. <https://eur-lex.europa.eu/eli/reg/2017/745/oj>, 2017. Accedido: 23-06-2026.
- [20] Vite. Vite documentation. <https://vite.dev/>, 2026. Accedido: 23-06-2026.
- [21] ZenHub. Zenhub help center. <https://help.zenhub.com/>, 2026. Accedido: 23-06-2026.